C++

Dr. Justin Furuness

Course Topics

- In this course we'll be going over C++ and Object Oriented Design!
- We'll also cover various tools that are critical for low-level developers (such as C++ devs):
  - The terminal, Git/GitHub, Virtual Environments, Testing (with PyTest), Vim/Emacs, Tmux, etc.
  - All of which are very critical for jobs these days
  - It would be very hard to get hired in my opinion if you don't know the basics of these tools
- Also, please just raise your hand or interrupt me with questions as we go, for this lecture and all others

Course Videos

- Laurent filmed an earlier version of this course
- Definitely was a HUGE inspiration as to what the course became today
- His videos are great! But they are in a different order as to what we'll be covering
- Additionally, I likely won't be making videos this term, as my time will be devoted towards building the material
- So make sure you show up to class!

Why Learn C++?

- C++ is one of the more powerful programming languages
- It allows for extremely fine-grained control of memory, performance, and design
- These qualities make it the go-to language for critical software systems.

Applications of C++

- Operating Systems, Compilers
- Games and Graphics
- Finance (HFT, financial systems) and Databases
- Large-scale Simulations, CUDA
- It's in most places where performance really matters

C++ vs Other Languages

- Python
  - Way higher level than C++, as we'll see
  - 10-100x slower for the same code in most cases
  - Generally considered faster to develop in
  - Errors easier to debug
- Java
  - Falling in popularity in industry
  - Worst of both worlds in comparision to Python/C++
  - Not as high level as Python, not as low level as C++
- Go
  - Google made this
  - It never really took off in the larger industry like these other languages
  - IMO industry is moving away from Go, preferring Rust

C++ vs Other Languages

- C
  - C++ is mostly a superset of C, i.e. C plus classes!
  - C lacks a lot of the abstractions that C++ provides
  - Still used where performance is extremely critical and/or code must be very lightweight, like with microcontrollers
- Rust
  - Mostly a subset of C++, with some new features for memory safety
  - Forces you to code in a non-object-oriented way (no classes)
  - Voted most popular language nine years in a row on stack overflow
  - A huge portion of industry is moving to this for a significant portion of the stack
  - Rust is hard to understand without first knowing C++!

C++ vs Other Languages

- Objective-C
  - formerly dominant in Apple ecosystem
  - largely replaced by Swift for app development.
- C#
  - Microsoft-backed, used in some Windows applications and Unity game engine.
  - These days, even Microsoft typically uses C++ instead of C#
  - Never was even close to as popular as C++
- So C++ really is a great language to learn!

C++ Dev vs Others

- This course is designed to help you on the career path towards a C++ Software Engineer
  - If you can master C++ and write good performant code, you'll be golden
- This is very different from other jobs
- Python developer
  - Generally the back-end, requiring learning vast amounts of libraries:
    - FastAPI, Django, Numpy, Pandas, Docker, etc.
- The same is true for front-end, infrastructure, etc.
  - An large breadth of knowledge is needed to be competitive

C++ Dev vs Others

- Salaries: typically much higher for C++ jobs
  - Other languages are simply easier to learn and more people know them
- C++ is also more resilient to AI than other languages
  - The faster code becomes, the less clean it gets in general and the harder it is for AI to follow
  - There's also a lot less code examples online compared to something like Python
- See `https://levels.fyi/2025` for salary data. Something to strive for!

Prerequisites

- You're supposed to have CSE3100, CSE2050, CSE1010, etc.
- That being said, students told me last semester that they nothing really stuck from these courses, especially when it comes to knowing C
- Additionally there is the break between semesters as well so plenty is forgotten
- I try to teach the course assuming little to no pre-requisite knowledge
  - I may even try to remove some of these prerequisites in the future
  - Be warned though - the course picks up speed rather quickly

C++ as a First Language

- For those coming from non-CS majors:
- I don't recommend learning C++ as your first language
- Other languages like Python or even JavaScript (ugh) are much easier
- C++ can get pretty in depth, and this class will dive deep on it for sure
- That being said, some people like learning lower level languages like C++ first
  - To each their own

Difficulty

- I also wanted to touch on the difficulty for this course
- I try to align my expectations of you with what is going to be expected when you go out and try to get a job
- For most, that means this will be a very hard class
  - To me it's worthwhile and you'll learn a lot of skills that are necessary for getting a job
  - But that does involve a considerable amount of hard work

Difficulty

- This is also a 3000 level course
- Expectations are going to be higher than other lower level courses
- If you fall behind, the expectation is that you show up to office hours and catch up
- For most, if you don't show up to class, or use AI on all the homework assignments, you'll likely fail the class
  - I know that's not the case with other classes, I'm just trying to set expectations early so that it's not a surprise

AI

- AI is amazing and there are some amazing AI tools out there
- Boris Cherny (created Claude Code at Anthropic) claims he hasn't typed code by hand in two months. 259 PRs in 30 days
- Ryan Dahl, The creator of Node.js and Deno likewise said "the era of writing code is over"
- Charlie Marsh (Astral, Ruff, UV Ty etc): "I think it's been over a month since I made a code change that didn't go through Claude Code"
- Every other major AI startup (XAI with Grok, Google with Gemini, OpenAI with ChatGPT) all are using claude-code as well

AI

- But to get hired - you must know more than the AI! Or else what's the point of hiring you instead of just prompting an AI?
- You must be able to direct AI based on your programming knowledge.
- Additionally, industry is not using ChatGPT like students are
  - They are using agentic models and tools like claude-code and/or cursor, generally in fairly specific ways
- Using ChatGPT isn't gaining you any employable skills imo, not even AI

AI

- I'm trying to teach you the building blocks of programming
- This way, when you encounter problems that AI can not solve, you can succeed
- Additionally, many problems AI can solve with direction, and you'll be able to give this direction
- You'll see this in the course project at the end of this course
  - Last semester many students used AI and they had to rewrite large portions manually
  - Good programmers think AI can do everything
  - bad programmers using AI just produces garbage and can't get it to work
- Learn this course well and when you add AI afterwards you'll be golden!

AI

- Therefore my recommendation is that you not use AI to complete assignments unless it's otherwise specified
- When you need help, personally I recommend using ChatGPT study mode
- Ask the AI to generate problems on the subject and walk you through them
- Then complete a problem from the AI without help
- Then try to complete the original problem
- This isn't a steadfast rule however, use your own judgment, it's just my recommendation
- I won't penalize people for AI, but imo you're just hurting yourself in the long run if you use it
  - Last semester, people got 100s on hws and straight zeroes on exams because they used AI as a crutch excessively
- Also don't forget, most interviews don't allow AI

Course Structure & Grading

- This will be the overall structure of the class in terms of grading/teaching:
- Lectures
- Homeworks (7%)
- Labs (7%)
- Quizzes (10%)
- Exams (60%)
- Course Project (16%)
- If you get an F on either the quizzes, the exams, or the course project, you fail the class.

Lectures

- Lectures are twice a week, totally 2.5hrs/wk
- You can always follow along with the slides but I strongly recommend coming to class
  - Last semester the people who showed up did very well
  - Most who followed along on their own struggled
  - Generally you'll for the first little bit, giving a false sense of confidence...
    - Then we'll move too quickly and you won't be able to catch up
- When you miss class or want to improve your understanding, come to office hours of myself or TAs
  - These will be posted when I have them from TAs

Homeworks

- Homeworks will be worth 7%
  - This likely will equate to about half a point per homework
  - This however could change depending on how many we get to
- They'll be released on Fridays and due the following Friday. There will be a one week grace period.

Labs

- Labs will be worth 7%
  - This likely will equate to about half a point per lab
  - This however could change depending on how many we get to
- They'll be released on Mondays and due on Friday (the same time as the homeworks). There will also be a one week grace period for labs.
- Labs are designed to be short enough to complete in lab
- Lab participation is not required, but there will be quizzes and potentially exams in labs

Labs and Homeworks: Extensions

- Last semester I ended up spending often multiple hours a day dealing with one-off requests, extensions, etc
- Now I'm teaching multiple sections, this is infeasible and is a waste of time
- To solve this problem, I've given a one week grace period for every single assignment
  - This means you can submit a week late and receive no penalty
- This also fixes the problem of people signing up two weeks in
- These assignments were completable within a single week last term
- There will be no extensions beyond this already one week extension for any reason whatsoever
- Complete the assignments when they come out
- Get help on the assignments during office hours
- If you're late, it's a zero. No exceptions.

Exams

- Three exams
- 20% each
- Short answer and coding based
- These will likely be a combination of some short answer problems
- This will be followed by a few coding problems, open-ended
  - You will actually program for this
- They'll be taken with the lockdown browser
- If you get an F average on the exams, you fail the class
- 2/12, 3/12, and the final exam are the exam dates

Quizzes

- Likely one each week
- 10% total
- Open-ended coding problems
  - You will actually program for this
- They'll be taken with the lockdown browser
- If you get an F average on the quizzes, you fail the class
- If you get a 90+% on an exam, all quizzes will automatically be a 100% until the next exam
- I will drop your lowest quarter of quizzes - this is specifically meant for absences

Course Project

- Worth 16% of grade
- This will take place during the latter half of the semester
- You'll be building a large codebase to simulate the internet with C++
- AI will be allowed - but it will likely not help too much
- Relatively low amount of starter code - you'll code this from start to finish
- If you get an F on the course project you fail the class

Communication

- Discord for all communication
- Username: first.last
- Invite: `https://discord.gg/9PkX6GjtvC`
  - Check huskyct for the code
- Office hours: TBD
- Note that ALL announcements will come through Discord. If you don't sign up, you will miss them.

Cheating

- The department is cracking down on cheating now
- If you cheat on anything:
  - You will get a zero for the course
  - I will report you for it
- In this class, you're really only cheating yourself
- How well you learn this material will likely directly impact how much money you make as a software engineer

Use the Resources Available

- Ask for help often
- Lecture, Office hours, TAs
- Peer Tutoring Center
- Discord
- ChatGPT Study Mode

Laptop

- Everyone is expected to have a laptop in all classes and labs
- Everyone is also expected to have a charger brought everywhere
- We'll get into how to configure your laptop later on, but for now just make sure you have one
- Additionally, if you are on a windows machine, before next class, you must have installed WSL2 (Windows subsystem for Linux 2)

Who Am I?

- Justin Furuness
  - Just call me Justin
- PhD in CS, BGP Security
- Undergrad, masters, PhD, all at UConn
- Worked at NeuralFrame and try to make this class applicable for industry
- Developed popular Python packages
- Code used worldwide and in many publications (NIST, others)
- Experience with PyBind11/C++ extensions
- Taught this class last term!

Questions?

- By the way, please just interrupt me any time there is a question

Hello World

- There's a lot of glue that we'll add throughout this course in terms of tooling
  - The terminal, Git/GitHub, Virtual Environments, PyTest, and a few others
- This will allow us to be able to compile, run, and test our code
- But first, let's go over some C++ basics!

Hello World

- Go ahead to `clientide.com` and select C++
- We're going to start off with writing a hello world program in C++
  - AKA a program that is going to print hello world
  - A program is a list of instructions that the computer follows from top to bottom
- Go ahead and start by writing the code below

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Hello World

- You can go ahead and run this code
- Congrats, you've completed your first C++ program!
- There's a lot going on here, but we don't know enough to be able to dig into most of it
- The most important part is that we can modify the lines inside of the curly braces to print and do different things!
- We're going to start there, and come back to this once we have a better understanding

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Hello World

- We'll just go over this code at a very high level
- `#include<iostream>` allows us to use cout and endl
- The main function is where every program starts
  - Everything inside the curly braces gets run
  - Indentation is optional, curly braces are what really matters

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Hello World

- Everything to the right of std::cout is what gets printed
- std::endl adds a new line (i.e. goes to a newline when printing)
- Any text that gets printed needs to be in quotes (single or double)
  - Caveat: Numbers don't need quotes

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Questions?

Comments

- First things first, in C++ you can create comments!
- Comments are a nice way to let readers know what your code is doing
- They're just notes to yourself or future readers, they affect nothing
- Remember code is read far more often than it is written, so make sure to comment your code when it makes sense!
  - Typically comments are only necessary when they help clarify something that isn't clear
  - Even though a more experienced C++ developer probably wouldn't need them in this case, we'll add them anyways just to showcase them

Comments

- In C++ There are two ways to create comments: a double slash, or /* */ for multiline comments
- Generally, comments go above what they are describing
- Rarely they can be on the side, if the comment is very short and it would be less clear above the line.

Hello World Example

```
/*
This is a multiline comment
About this piece of code
*/

#include <iostream>

int main() {
    // This line prints out Hello World!
    std::cout << "Hello world!" << std::endl;
    return 0; // success!
}
```

Variable Declaration and Assignment

- In C++ we can also declare (aka create) a variable
- Variables are basically stored data that has a name
- C++ is strongly typed, so all variables must have types
- Let's take a look at an example for how to create a variable holding the number 100 and print it

Variable Example

```
#include <iostream>

int main() {
    int num_students;  // Declaring the variable
    num_students = 100;  // Assigning to the variable
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- First we declare the variable num_students
- Any time that we declare a variable, we must include what type the variable is
- num_students has the type int, which stands for integer, which means this is a number
  - We'll dive deeper into types later on

Variable Example

```
#include <iostream>

int main() {
    int num_students;  // Declaring the variable
    num_students = 100;  // Assigning to the variable
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- Directly below our declaration, we assign 100 to our variable
- Here the equals is the assignment operator, NOT equality
- The assignment operator is taking the value 100, and storing it in the variable named num_students
- The assignment operator *always* goes right to left

Variable Example

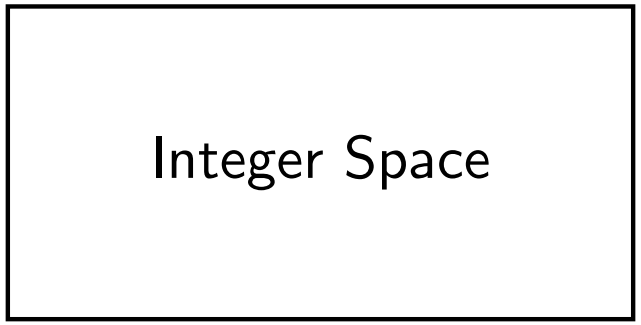
```
#include <iostream>

int main() {
    int num_students;  // Declaring the variable
    num_students = 100;  // Assigning to the variable
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- When we declare a variable, C++ gets space from our computer to store the variable
- The space it takes from your computer is in the computers memory, or RAM (Random Access Memory)
- The amount of memory that the computer requests is exactly the size of the variable it is storing
- If we declare an integer variable, C++ gets enough memory to store an integer
- Later C++ can set the integer in that space in memory to be whatever you wish

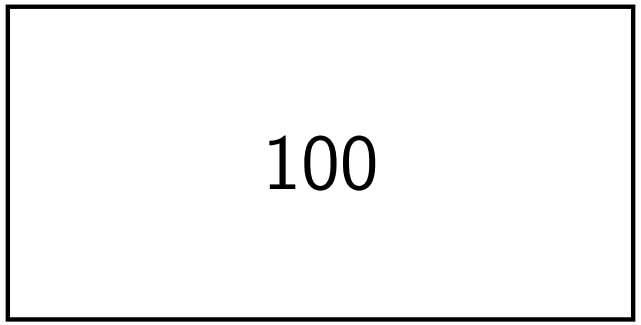
Variable Declaration and Assignment




```
int num_students;
```

C++ has reserved space in memory for an integer, but no value has been stored yet.

Variable Declaration and Assignment




```
num_students = 100;
```

The value 100 is now stored inside the memory space associated with `num_students`.

Variable Declaration and Assignment

- We can also both declare a variable, and assign a value to it, on the same line
- There is no difference between declaring and assigning a value to a variable on the same line versus separate lines
- Typically in C++ people want concise code, so they like to do both at once whenever possible

Variable Example

```
#include <iostream>

int main() {
    // Declaring a variable and assigning to it
    int num_students = 100;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Semicolons

- Notice how at the end of each line there is a semicolon
- That's because this C++ program is comprised of many statements
- A statement in C++ is a command that instructs the program to perform a specific action
- We need to let the C++ compiler that runs your program know when a statement ends
- A statement doesn't end on a new line, it ends with a semicolon!

Semicolons

- Because statements end with a semicolon and not a new line...
- If we wanted, we could write many statements on one line!
  - Although that would certainly be a lot less clear

Variable Example

```
#include <iostream>
int main() {
    int num_students=100;std::cout<<"Students="<<
        num_students<<std::endl;return 0;
}
```

Variable Declaration and Assignment

- In C++ variables can change as your program runs
- In other words, variables can "vary"
- When we are changing a variable, we can't change it's type, only the value of the variable
  - So an int variable must remain an int variable
  - Later on we'll go over special ways to convert between types, but let's leave this to the side for now
- Take a look at this program now, and you can see that it outputs 5, not 100!

Variable Declaration and Assignment

- Here we set the variable to be something different

Variable Example

```
#include <iostream>

int main() {
    int num_students = 100;
    num_students = 5;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- We can even make variables equal to other variables!

Variable Example

```
#include <iostream>

int main() {
    int num_students = 100;
    int other_students = 200;
    num_students = other_students;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Naming Conventions

- In terms of what to call your variables in C++
- Variable names can contain letters, digits, and underscores ONLY
  - No whitespace or special characters
  - Digits are also an odd choice to put in variables
- Names must begin with a letter or an underscore
- Certain keywords that the compiler already has (such as int, class, etc) can't be used as variable names
- Generally, snake case works well
  - This is lowercase with underscores to separate words
  - Such as with num_students

Types

- When you declare a variable, you must also declare the type of the variable
- There are LOTS of types in C++, but here are some of the more common ones:
  - int (for whole numbers)
  - double (for decimals)
  - bool (for true or false)
  - char (for characters)
- We'll dive deeper into all of these types later on
- For now, let's just practice creating some of them!

Types

Variable Example

```
#include <iostream>

int main() {
    int num_students = 100;
    double grade_average = 98.6;
    // A character is a single character, "Hello" is many
        characters
    char first_initial = 'j';  // Chars must use single
        quotes!
    std::cout << "Average=" << grade_average << std::endl;
    return 0;
}
```

Types

- Boolean values can either be true or false, or 1 or 0
- C++ will automatically convert the 1 or 0 to a true/false
- In fact, any non-zero number converts to true
- They are always printed as 1 or 0 to be backwards compatible with C

Variable Example

```
#include <iostream>
int main() {
    bool pass_the_class = true;
    bool fail_the_class = false;
    bool you_guys_are_the_best = 1;
    fail_the_class = 0;
    std::cout << "Pass=" << pass_the_class << std::endl;
    return 0;
}
```

Questions?

Arithmetic Expressions

- We can also do all the things your calculator can do with numbers!
- We can add, subtract, multiply, divide, use parenthesis, use exponents, and more!

Variable Example

```
#include <iostream>
int main() {
    int total_cpp_students = 100;
    int total_python_students = 200;
    int total_students = total_cpp_students +
        total_python_students;
    std::cout << "Students=" << total_students << std::endl;
    return 0;
}
```

Arithmetic Expressions

- We can also do all the things your calculator can do with numbers!
- We can add, subtract, multiply, divide, use parenthesis, and more!

Variable Example

```
#include <iostream>
int main() {
    int total_cpp_students = 100;
    int total_python_students = 200;
    int total_students = total_cpp_students +
        total_python_students;
    std::cout << "Students=" << total_students << std::endl;
    return 0;
}
```

Arithmetic Expressions

- Here's an example for how to subtract

Variable Example

```
#include <iostream>
int main() {
    int total_cpp_students = 300 - 200;
    std::cout << "Students=" << total_cpp_students << std::
        endl;
    return 0;
}
```

Arithmetic Expressions

- We can also multiply

Variable Example

```
#include <iostream>
int main() {
    int assignments_per_student = 5;
    int num_students = 100;
    int total_assignments = num_students *
        assignments_per_student;
    std::cout << "Assignments=" << total_assignments << std
        ::endl;
    return 0;
}
```

Arithmetic Expressions

- We can also divide

Variable Example

```
#include <iostream>
int main() {
    int total_assignments = 500;
    int num_students = 100;
    int assignments_per_student = total_assignments /
        num_students;
    std::cout << "Assignments/Student=" <<
        assignments_per_student << std::endl;
    return 0;
}
```

Arithmetic Expressions

- We can also make more complicated math statements with parenthesis

Variable Example

```
#include <iostream>
int main() {
    int value = (5 * 4) / (2 * 5);
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

Modulo

- The modulo operator is very useful
- This leaves us only with the remainder

Variable Example

```
#include <iostream>
int main() {
    int value = 10 % 4;
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

Modulo

- One common use case for modulo is to determine if a number is even
- For example if a number divided by 2 has no remainder, it is even

Variable Example

```
#include <iostream>
int main() {
    int value = 10 % 2;
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

Questions?

Type Conversions

- Let's take a look at something slightly more complex
- What if you have two integers that don't divide evenly?

Variable Example

```
#include <iostream>
int main() {
    int numerator = 10;
    int denominator = 4;
    int value = numerator / denominator;
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

Type Conversions

- Here you have what's called integer division!
- Normally this would be 2.5
- But the value can't be 2.5 since that is not an integer!
- Instead, C++ gets rid of everything after the decimal
  - Often referred to as "truncating the decimal"
- Integer division occurs when both numerator and denominator are integers, or the final result is an integer

Variable Example

```
#include <iostream>
int main() {
    int numer = 10;
    int denom = 4;
    int val = numer / denom;
    std::cout << "Value=" << val << std::endl;
    return 0;
}
```

Type Conversions

- To fix this, we can convert *either* type to a double
- Our final result, the value, *must* also be a double

Variable Example

```
#include <iostream>
int main() {
    double n = 10;
    int d = 4;
    double v = n / d;
    std::cout << "Value=" << v << std::endl;
    return 0;
}
```

Variable Shortening

- You may also notice that I've been shortening the variable names as we went
- This is a very common practice in programming in general
  - But *definitely* in lower level languages like C++ or C
- Generally you want your variable names to be descriptive enough so that others can understand
  - n, d, v may be a bit too short in this case!
  - The program loses a bit of readability
  - However, in some places, single letter variables may improve readability!

Variable Shortening

Variable Example

```
#include <iostream>
int main() {
    double n = 10;
    int d = 4;
    double v = n / d;
    std::cout << "Value=" << v << std::endl;
    return 0;
}
```

Questions?

Assignment operators

- We've already been practicing with one assignment operator, the "=" sign
- C++ Gives us several others that we can use for shortcuts
- For example, let's take this very common pattern

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int just_signed_up = 20;
    // Student enrolls here!
    num_students = num_students + just_signed_up;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- Instead, we can shorten this with the "+=" assignment operator

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int just_signed_up = 20;
    // Student enrolls here!
    // num_students = num_students + just_signed_up;
    num_students += just_signed_up;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- We can do the same with -=

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int just_dropped = 20;
    // Student drops the class here!
    // num_students = num_students - just_dropped;
    num_students -= just_dropped;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- And *=

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int classes_merged = 2;
    // Classes merge here
    // num_students = num_students * classes_merged;
    num_students *= classes_merged;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- And /=

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int split_class = 2;
    // Classes split in half here
    // num_students = num_students / split_class;
    num_students /= split_class;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Questions?

Increment/Decrement

- Often times we are just adding or subtracting one from a given number
- C++ gives us a really easy way to do this
  - Even shorter than the "+=" and "-="

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    // Student enrolls here!
    // num_students = num_students + 1;
    // num_students += 1;
    num_students++;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Increment/Decrement

- And of course C++ provides the decrement shortcut for subtraction

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    // Student drops here!
    // num_students = num_students - 1;
    // num_students -= 1;
    num_students--;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Increment/Decrement

- Increment and decreemnt can be done inline
- If it is before, increment runs before the statement executes.
- Otherwise increment/decrement runs after the statement executes.
- This is an extremely common way to reduce lines of code.

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    std::cout << "Students=" << num_students++ << std::endl;
    std::cout << "Students=" << num_students << std::endl;
    num_students = 100;
    std::cout << "Students=" << ++num_students << std::endl;
    return 0;
}
```

Common Errors

- Forgotten semicolons
- Misspelling variable names
- Using variables before declaring them

Variable Example

```
#include <iostream>
int main() {
    num_students = 100
    // Student drops here!
    num_student--;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Questions?

In class exercises

- Write a hello world program

In Class Exercises

In Class Exercises

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

In class exercises

- Set a variable for total students equal to 100
- Set a variable for new students equal to 20
- Add the new students to the total students using an assignment operator
- Decrement total students by 1
- print the remainder of the total students when divided by 2
    - This will indicate if the total number of students are divisible by 2

In Class Exercises

```
#include <iostream>
int main() {
    int num_students = 100;
    int new_students = 20;
    num_students += new_students;
    num_students -= 1;
    std::cout << num_students % 2 << std::endl;
    return 0;
}
```

In class exercises

- Get the average grade given two scores of 90 and 93
  - To get the average: Add the scores together, then divide by the total number of scores

In Class Exercises

In Class Exercises

```
#include <iostream>
int main() {
    double average = (90 + 93) / 2.0;
    std::cout << average << std::endl;
    return 0;
}
```

In class exercises

- Store the first letter of your name as a character and print it

In Class Exercises

```
#include <iostream>
int main() {
    char first_initial = 'j';
    std::cout << first_initial << std::endl;
    return 0;
}
```

In class exercises

- Store store a boolean value and print it

In Class Exercises

```
#include <iostream>
int main() {
    bool passing = true;
    std::cout << passing << std::endl;
    return 0;
}
```

Expanding Our Memory Model

- Especially for C++ code (which often must be very fast), it's very important to understand why and how memory works in C++
- This will allow us in the future to optimize our code
  - Both for speed
  - And for memory

Expanding Our Memory Model

- Remember that your computer is merely a bunch of transistors
  - If you don't know what a transistor is, that's okay
- These transistors have current (electricity) flowing through them
- When the voltage on a transistor is low, it's represented as a zero
- When the voltage on a transistor is high, it's represented as a one
- From a high level, you can kind of think of your computer as a bunch of switches that are either on (represented as 1) or off (represented as 0)

Expanding Our Memory Model

- When you put a bunch of these ones and zeroes together, you get what's called binary code
- In other words, lines of code that are ones and zeroes, simply telling your computer which switches to turn on or off
- This is how all coding works under the hood
- When we click this run button here, our C++ code is being converted to ones and zeroes before being run

Expanding Our Memory Model

- So then how does memory work on a computer?
- Well, every variable gets stored as zeroes and ones
- The term for this is binary
- For example, numbers can be stored in binary as a binary number, as zeroes and ones
- So how do binary numbers work?

Expanding Our Memory Model

- Instead of counting in base 10 like we normally do, we can count in base 2
- Normally for base 10, every digit represents 10 to the next power
- 257 represents `10^2 * 2 + 10^1 * 5 + 10^0 * 7` in base 10
  - Base10 is what we normally count in
- We can do the same but in binary

Expanding Our Memory Model

- Let's start with a type of uint8_t
- This is a type of C++ that sets aside 8 of these "switches"
- Each switch represents what we call a "bit" on our computer
- And your computer groups these bits together in groups of 8 called a "byte"
- A "byte" is just a group of 8 bits
- So for uint8_t, this is simply a number that takes up one byte, or 8 bits of space

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number 0, which in binary might be represented as 00000000

Bit Flipped	0	0	0	0	0	0	0	0
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	0	0

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 1, which in binary might be represented as 00000001

Bit Flipped	0	0	0	0	0	0	0	1
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	0	1

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 2, which in binary might be represented as 00000010

Bit Flipped	0	0	0	0	0	0	1	0
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	2	0

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 3, which in binary might be represented as 00000011

Bit Flipped	0	0	0	0	0	0	1	1
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	2	1

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

End of day 1

In class exercise

- How do you store 10 in binary as a uint8_t?
- How do you store 21 in binary as a uint8_t?

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 10, which in binary might be represented as 00001010

Bit Flipped	0	0	0	0	1	0	1	0
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	8	0	2	0

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 21, which in binary might be represented as 00010101

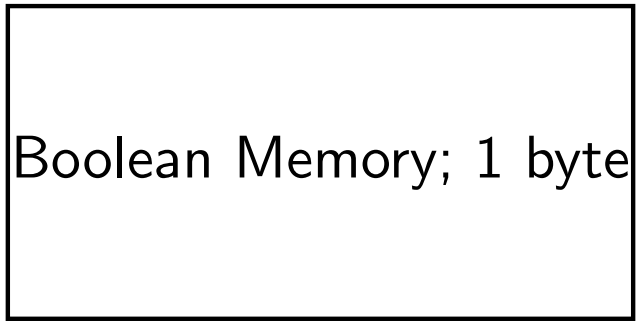
Bit Flipped	0	0	0	1	0	1	0	1
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	16	0	4	0	1

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Expanding Our Memory Model

- Your computer has a limited amount of these bits
- Remember that these bits are grouped into 8 called bytes
- For example, your iphone might have 16 gigabytes
  - That's it! No more space!
- All of the types that we've discussed thus far are also represented in binary (ones and zeroes)
- For example, take a boolean type
- A boolean type for example, only needs a single bit, since it's either true (1) or false (0)
- However, because your computer always groups 8 bits together, you have a minimum of one byte of space for every type
- So one boolean gets one byte of space reserved for it

Expanding Our Memory Model




```
bool passing;
```

When this line of code runs, C++ reserves space in memory for exactly one byte for this boolean value, but no value has been stored yet. While the boolean value needs only a single bit, it must occupy the entire byte of space

Expanding Our Memory Model

- Likewise, letters, stored as characters (the char type), are also represented in binary
- These have one byte associated to them
- Each binary number gets converted to a letter
- This conversion system is known as ASCII
- For example, a capital 'A' is stored as the number 65
- A lowercase 'a' is stored as the number 97

Expanding Our Memory Model

- Integers can also be stored in this way
- Integers are represented in binary (1s and 0s)
- In C++ integers are 32 bits, (32 1s and 0s)
  - How many bytes is that?
- So how do we go from 32 bits to an actual number?

Expanding Our Memory Model

- 32 bits are simply four bytes back to back
- The way that memory works is that you MUST have these bytes placed next to each other
- You can skip memory blocks
- This is called "contiguous memory"

Expanding Our Memory Model

- Intuitively, you may be thinking that 32 bits for the number one can be stored as:
  - |0000 0000 |0000 0000 |0000 0000 |0000 0001 |
  - (here each pipe is representing a block of space)
- And it can... on big endian systems
- But typically most computers actually arrange the bytes in reverse order
- This is called little endian
- So the number one would be stored as:
  - |0000 0001 |0000 0000 |0000 0000 |0000 0000 |
- While I cover it here just so we are aware of how it works, most of the time you don't need to think about endianness unless you're working with raw bytes, files, or networks.

Binary Bits and Their Values

- Of course, for a 32 bit number, the maximum size that the number can be is 2^32 - 1
- This is when all the "switches" are turned on
- This is when all the bits are set to 1
- Every type has a maximum and a minimum size
  - We'll dig into the implications of this in a moment

Binary Bits and Their Values

- If we say that the integer is "signed", that means that the first bit represents the sign
- 0 For positive
- 1 for negative
- This can get a bit complicated as to how these binary numbers are represented, so we won't go into the binary for this since it's not really necessary to know
- It's just important to know that for signed numbers, the maximum size has one bit less
- So for an integer, the maximum size for a "signed" integer (the default) is 2^31 - 1
  - This is in contrast to an unsigned integer where the default max size is 2^32 - 1

Questions?

Expanding Our Memory Model

- Integers take up 4 bytes, aka 32 bits, on most machines
- That means when you declare and create an integer, your computer is reserving 4 bytes or 32 bits of space for that integer
- In C++ since an integer is a "signed" type, the first bit is reserved for indicating whether a number is positive or negative (for example, negative 4, or positive 4)
- That means the maximum size of that integer is actually the number when the remaining 31 bits are set to 1
- In other words, the maximum size of an signed integer (keyword: int) is 2,147,483,647
  - 2,147,483,647 in binary is 01111111111111111111111111111111

Expanding Our Memory Model

int num_students (4 bytes) = 2147483647

```
int num_students=2147483647;
```

When this line of code runs, C++ reserves **4 bytes (32 bits)** of memory to store the integer `num_students`, and we set the value to be 2147483647 (the maximum integer value).

Overflow Error

- So then what happens when we add one to the maximum value an integer can be?
- The number can not grow to 33 bits...
- So it "overflows" and actually wraps around!
- Think: Just like what happens when you add one hour to a twelve hour clock!
  - Adding an hour to 6PM, no problem. 7PM, no problem. All the way to 11PM, no problem. But if you try to add one hour at 12PM, you wrap back around to 1AM!
  - The same is true for numbers in C++!
- When are add 1 to the most positive integer, 2147483647:
  - It wraps around to the most negative integer, -2147483648!
- This is called an overflow error

Overflow Error

- In C++ however, overflows aren't always that nice
- Sometimes, overflow errors result in undefined behavior
- That means your program can start doing wacky stuff that doesn't make any sense
- It's also undefined, meaning different runs it may do different things
- It can be very hard to debug
- On most machines you'll see wraparound, but the standard does not guarantee this
    - The compiler is allowed to do anything

Overflow Error

- Let's take a look at this in our code!
- We can cause an overflow error to happen!
- Now the printed value is actually wrapping around and is the most negative integer

Example

```
#include <iostream>
int main() {
    int num = 2147483647;
    std::cout << num + 2 << std::endl;
    return 0;
}
```

Overflow Error

- To avoid this, it's really important to ensure that all numbers fit within their specified range for that specific type
- There are many numeric types just for these situations
- Also see these unsigned types, which assume that the number is always positive
  - Then all bits are used for the number itself (there are no sign bits)

Type	Size	Minimum Value	Maximum Value
`bool`	1 byte	0	1
`char`	1 byte	-128	127
`unsigned char`	1 byte	0	255
`short`	2 bytes	-32,768	32,767
`unsigned short`	2 bytes	0	65,535
`int`	4 bytes	-2,147,483,648	2,147,483,647
`unsigned int`	4 bytes	0	4,294,967,295
`long long`	8 bytes	-9.22×10^{18}	9.22×10^{18}
`unsigned long long`	8 bytes	0	1.84×10^{19}

Overflow Error

- Here's a similar table for dealing with what's called **floating point** numbers
  - In other words, numbers with a decimal point
- Floating point values can also overflow or underflow
- These types sacrifice precision for a larger range
- They may also suffer from **rounding errors**...
  - We'll come back to this later

Type	Size	Minimum Value	Maximum Value
`float`	4 bytes	$\approx 1.18 \times 10^{-38}$	$\approx 3.40 \times 10^{38}$
`double`	8 bytes	$\approx 2.23 \times 10^{-308}$	$\approx 1.80 \times 10^{308}$
`long double`	16 bytes	$\approx 3.36 \times 10^{-4932}$	$\approx 1.18 \times 10^{4932}$

Overflow Error

- Absolutely no need to memorize these tables
- Just make sure to understand these concepts:
  - Overflow
  - Signed vs unsigned
  - Code is converted to binary which tells your computer what to do
  - Numbers are stored in binary and take up specific sizes in bytes
  - C++ does not reserve less size than a byte of space in memory
    - So even a boolean (1 bit) takes up an entire byte of memory
  - Each byte is 8 bits
  - Each bit represents a single switch on your computer

Questions?

Arrays

- What about when you want to be able to store many items at once?
- In other words, what if you want to have 20 student scores or 1000?
- It's a bit silly if we have to store separate variables
- Instead, we can just put them in what's called an array
- An array is a collection of values of the same type
- All of the values are stored right next to each other in memory
- Each value in the array can be accessed using a number called an index

Arrays

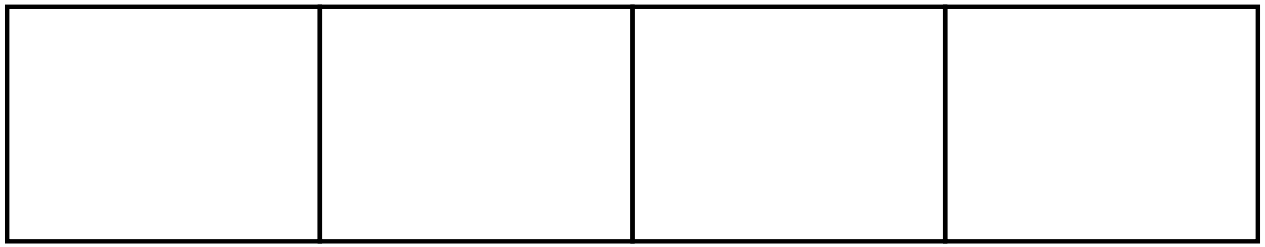
- You can create an array like so:
- Instead of int scores -> int scores[4]
- That tells C++ to grab four blocks of memory for integers
- Remember each integer is typically 4 bytes
- In total that means this array will take 16 bytes of space

Example

```
#include <iostream>
int main() {
    int scores[4];
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Arrays

int scores[4];



(4 bytes)	(4 bytes)	(4 bytes)	(4 bytes)
Index 0	**Index 1**	**Index 2**	**Index 3**

An array of 4 integers reserves **4 contiguous blocks of memory**, with each block taking **4 bytes**. Currently there is nothing in the array.
Note: Arrays are zero indexed in C++! That means we start counting from zero. So index/block zero, one, two....

Arrays

- When we want to set an item at a specific block in this array, we can specify the index and do so with the following:

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[0] = 90;
    scores[1] = 80;
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Arrays

int scores[4];
scores[0] = 90;
scores[1] = 80;

90	**80**		
(4 bytes)	(4 bytes)	(4 bytes)	(4 bytes)
Index 0	**Index 1**	**Index 2**	**Index 3**

Arrays

- We can also get the value at an index by using the same bracket syntax
- Example below

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[0] = 90;
    scores[1] = 80;
    int second_value = scores[1];
    std::cout << scores[0] << std::endl;
    return 0;
}
```

Arrays

- But what happens if we print one of the array slots that has not been filled yet?
- C++ will just read whatever 1s and 0s the computer already has there and try to turn it into an integer!
- This will result in undefined behavior and errors!

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[0] = 90;
    scores[1] = 80;
    std::cout << scores[2] << std::endl;
    return 0;
}
```

Arrays

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[5] = 80;
    std::cout << scores[4] << std::endl;
    return 0;
}
```

Arrays

- And what if we store a value at index 5? Even though there is no index 5 in the array?
- We will write to memory we do not own!
- This will write memory to outside of the array, clobbering other parts of the program while the program is running
  - It could potentially overwrite something very important!
  - Resulting in memory corruption, potentially leading to undefined behavior!
  - Not good!
- Unlike other programs like Python, C++ will often let this happen!
  - You must catch these bugs (errors) yourself! Be careful!

Arrays

- You can also set an array using curly braces at the time the array is created
- You can't declare the array, and later on set it using curly braces - it all must be done at once

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80, 70, 60, 100};
    std::cout << scores[1] << std::endl;
    return 0;
}
```

Arrays

- When you use curly braces to initialize your array, whatever isn't specified is defaulted to zero

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80}; // All other indexes are now
        zeroes!
    int other_scores[5] = {}; // Everything is zero!
    std::cout << scores[1] << std::endl;
    return 0;
}
```

Arrays

- Arrays also have a fixed size
- They are not dynamic
- The code must know this size in advance, it can't be a number that can change
- Additionally, once you've made the array, you can not later change it's size

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80, 70, 60, 100};
    std::cout << scores[1] << std::endl;
    return 0;
}
```

Arrays

- Another quirk of arrays: They do not keep track of their size!
- You as the programmer must keep track
- One "hack" to do this: Take the size in memory of the array, and divide it by the size in memory for a single block of that array
- You'll often see this used by C++/C programmers

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80, 70, 60, 100};
    int size = sizeof(scores) / sizeof(scores[0]);
    int also_size = sizeof(scores) / sizeof(int);  // Less
        dynamic
    std::cout << size << std::endl;
    return 0;
}
```

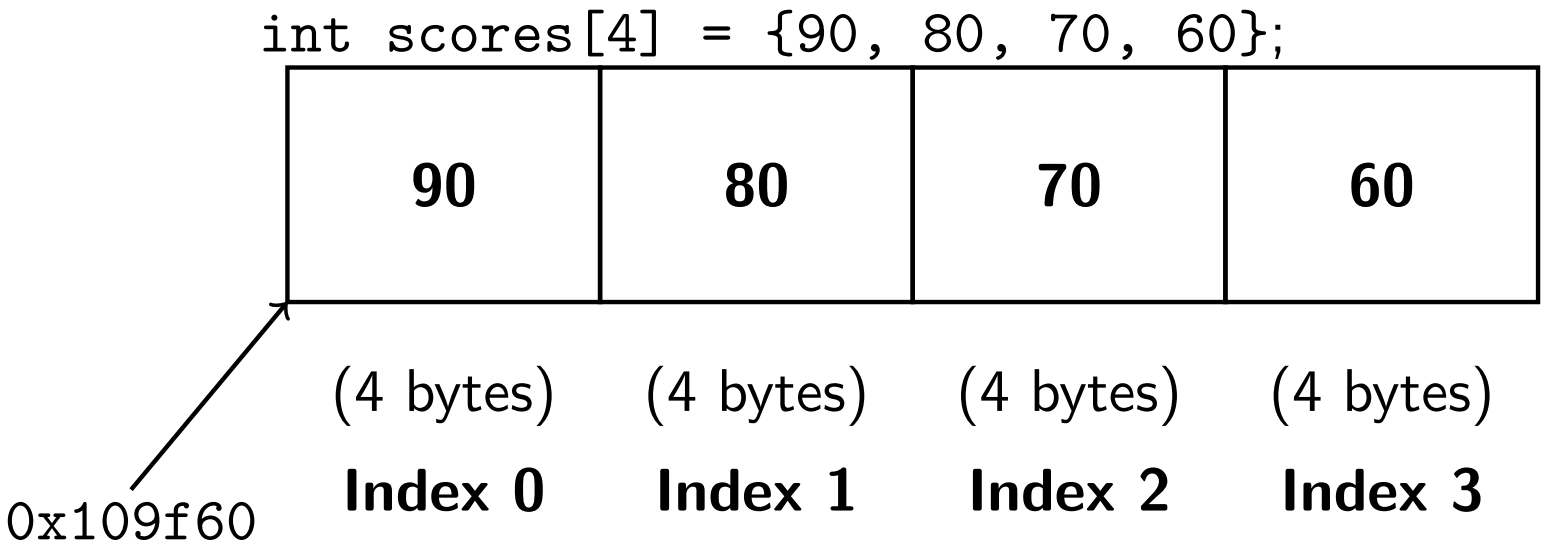
Arrays

- Lastly, you can print out the memory address of an array!
  - In other words, we can print out the exact location in memory where the array begins
- An array's address is the same as the address of its first element
- On most modern systems, memory addresses are 64-bit numbers

Example

```
#include <iostream>
int main() {
    int scores[4] = {90, 80, 70, 60};
    std::cout << scores << std::endl;
    return 0;
}
```

Arrays



- An array of 4 integers reserves **4 contiguous blocks of memory**, with each block taking **4 bytes**.
- We can get the beginning of the array, the address, by printing the array
- We can determine the size of the array with sizeof(scores) / sizeof(scores[0])

Arrays

- We can create an array of any type, not sure integers
- Let's create an array of booleans for example

Example

```
#include <iostream>
int main() {
    bool has_drivers_license[5] = {true, false, true, true,
        false};
    std::cout << has_drivers_license[1] << std::endl;
    return 0;
}
```

Questions?

In Class Exercise

- Create an array to store grades of 90.5, 89.6, 98.7 on one line
- Change the 89.6 to a 97 after the initial array creation
- Print the first grade in the array

In Class Exercise

- Create an array to store grades of 90.5, 89.6, 98.7 on one line
- Change the 89.6 to a 97 after the initial array creation
- Print the first grade in the array

Example

```
#include <iostream>
int main() {
    double grades[3] = {90.5, 89.6, 98.7};
    grades[1] = 97;
    std::cout << grades[1] << std::endl;
    return 0;
}
```

Character Arrays and C style strings

- Recall our hello world program
- Remember that we can print hello world using double quotes
- This is a C-style string, also in this case it is a string literal
- But what is the type of this "Hello World" C-style string?
  - And how can we store this in a variable?

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Character Arrays and C style strings

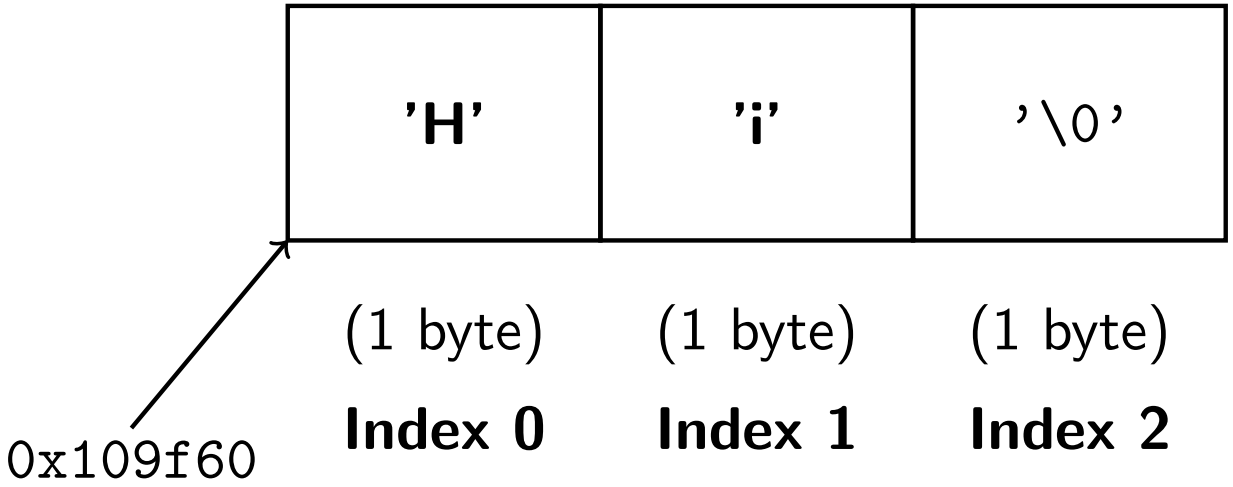
- "Hello World", under the hood, is actually stored as a character array!
- At the end of this character array is what's known as a null byte, i.e. the character '\0'
- The null byte lets C++ know:
  - "We've reached the end of our array, stop printing"
- The null byte is represented as a backslash zero, it's a special character
- Let's dive into why we need a null byte and expand on our memory model a bit more

Example

```
#include <iostream>
int main() {
    char message[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    std::cout << message << std::endl;
    return 0;
}
```

Arrays

```
char message[3] = {'H', 'i', '\0'};
```



- An array of 3 characters reserved **3 contiguous blocks of memory**, with each block taking **1 byte**.
- We can get the beginning of the array, the address, by printing the array
- We can determine the size of the array with sizeof(message) / sizeof(message[0])
- When printing, C++ will start at the beginning address of the array, and proceed to the final null byte

Character Arrays and C style strings

- Recall that C++ doesn't store the size of the array
- C++ also does store the beginning address of the array
- When printing, C++ goes block by block, starting at the beginning, until it reaches the null byte
- The null byte is how C++ knows it's reached the end
- The null byte in the character array is specifically what turns the character array into a C-style string that can be printed
- We can move the null byte further up and the printing will stop there

Example

```
#include <iostream>
int main() {
    char message[6] = {'H', 'e', '\0', 'l',  'l', 'o'};
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C style strings

- If we don't have a null byte, this results in undefined behavior!
- C++ may keep reading and printing from memory outside of the array!
- It may continue reading and printing until it finds a random null byte on your computer
  - Or do something else entirely! It's undefined!

Example

```
#include <iostream>
int main() {
    char message[6] = {'H', 'e', 'l', 'l', 'o', 'o'};
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C-Style Strings

- When we write `"Hello"`, C++ treats it as a **string literal**
- A string literal is stored as a **character array** ending with a null byte (`'\0'`)
- When we assign it to a character array, all characters *including the null byte* are copied

Example

```
#include <iostream>
int main() {
    char message[6] = "Hello";
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C-Style Strings

- We do **not** need to manually specify the size of a character array
- If we leave the size empty, the compiler will figure it out for us
- The compiler automatically:
  - Counts the characters in the string literal
  - Adds space for the null byte ('\0')
  - Allocates the correct amount of memory

Example

```
#include <iostream>
int main() {
    char message[] = "Hello";
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C-Style Strings

- And of course, just like in a a normal array, we can access any index
- We can print any index, or change any index

Example

```
#include <iostream>
int main() {
    char message[] = "Hello";
    message[0] = 'J'; // Recall: characters require single
        quotes!
    std::cout << message << std::endl;
    return 0;
}
```

Questions?

In Class Exercise

- Store your name in a character array
- Change the first initial of your name to be a T
- Print your name

In Class Exercise

- Store your name in a character array
- Change the first initial of your name to be a T
- Print your name

Example

```
#include <iostream>
int main() {
    char name[] = "Justin Furuness";
    name[0] = 'T';
    std::cout << name << std::endl;
    return 0;
}
```

In Class Exercise

- Create a C-Style string "Hello" using curly braces in a variable called msg
- Print msg

In Class Exercise

- Create a C-Style string "Hello World" using curly braces in a variable called msg
- Print msg

Example

```
#include <iostream>
int main() {
    char msg[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    std::cout << msg << std::endl;
    return 0;
}
```

Output with std::cout

- The next part of our program that we need to understand is `std::cout`
- `cout` stands for **character output**
- It represents a stream of data that gets printed to the terminal
- Anything sent to `std::cout` will appear on the screen

Example

```
#include <iostream>
int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

Output with std::cout

- The << operator is called the **stream insertion operator**
- It sends data into the output stream
- You can think of it as:
  - "Send this value to the screen"
- The data flows from right to left into `std::cout`

Example

```
std::cout << "Hello World";
```

Output with std::cout

- We can send *multiple* values to `std::cout` in one line
- Each value is sent using another <<
- The output happens in order, from left to right

Example

```
#include <iostream>
int main() {
    int students = 100;
    std::cout << "Students: " << students << std::endl;
    return 0;
}
```

Output with std::cout

- Notice how we can mix text and variables
- Strings must be in quotes
- Variables are written without quotes
- C++ automatically figures out how to print each type

Example

```
std::cout << "Answer = " << 42 << std::endl;
```

Output with std::cout

- `std::endl` means **end line**
- It does two things:
  - Moves the cursor to a new line
  - Flushes the output buffer
- For now, just think of it as "go to the next line"

Example

```
std::cout << "Hello" << std::endl;
std::cout << "World" << std::endl;
```

Output with std::cout

- We can print many different types using `std::cout`
- Including:
  - Integers
  - Doubles
  - Characters
  - Booleans
  - C-style strings

Example

```
#include <iostream>
int main() {
    int i = 10;
    double d = 3.14;
    char c = 'A';
    bool b = true;

    std::cout << i << " " << d << " " << c << " " << b <<
        std::endl;
    return 0;
}
```

Common std::cout Mistakes

- Forgetting << between values
- Forgetting quotes around text
- Using single quotes for strings
- Misspelling `std::cout`

Incorrect Example

```
std::cout "Hello World";
```

Questions?

In Class Exercise

- Print the following on one line:
  - The text `"Total Students:  "`
  - The number 250
- Then print a new line

In Class Exercise

Solution

```
#include <iostream>
int main() {
    std::cout << "Total Students: " << 250 << std::endl;
    return 0;
}
```

Functions

- So far, all of our code has lived inside `main`
- As programs grow, this quickly becomes hard to manage
- We need a way to:
  - Break code into smaller pieces
  - Reuse logic
  - Make programs easier to read and reason about
- This is where **functions** come in

Functions

- A function is a named block of code
- It performs a specific task
- We can:
  - Define a function once
  - Call it as many times as we want
- `main` is actually just a function!

Functions

- Let's look at the basic structure of a function
- Every function has:
  - A return type
  - A name
  - A list of parameters (possibly empty)
  - A body (inside curly braces)

Functions

General Function Form

```
return_type function_name(parameter_type parameter_name) {
    // function body
}
```

Functions

- Let's start with a very simple function
- This function takes no inputs and returns nothing
- The keyword `void` means "returns nothing"

Example

```
#include <iostream>

void say_hello() {
    std::cout << "Hello!" << std::endl;
}

int main() {
    say_hello();
    return 0;
}
```

Functions

- When we **call** a function:
  - The program jumps to the function
  - Runs the code inside it
  - Then returns back to where it was called
- Functions do *not* run unless they are called

Functions

- Functions can also take inputs
- These inputs are called **parameters**
- Parameters are variables local to the function

Example

```
#include <iostream>

void print_number(int x) {
    std::cout << "!" << x << "!" << std::endl;
}

int main() {
    print_number(42);
    return 0;
}
```

Functions

- Here, `x` receives a **copy** of the value passed in
- This is called **pass by value**
- The function gets its own copy
- Changing `x` does *not* change the original value

Functions (Pass by Value)

Example

```
#include <iostream>

void change_value(int x) {
    x = 100;
}

int main() {
    int num = 5;
    change_value(num);
    std::cout << num << std::endl;
    return 0;
}
```

- Output is still 5
- The function only modified its own copy

Functions

- Functions can also return values
- The return type specifies what kind of value is returned
- We use the `return` keyword to send a value back

Example

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);
    std::cout << result << std::endl;
    return 0;
}
```

Functions

- When a function returns:
  - The function immediately stops running
  - The returned value replaces the function call
- Think of it as:
  - `add(3, 4)` becomes 7

Functions

- Functions can take multiple parameters
- Each parameter must have a type
- Parameters are separated by commas

Example

```
double average(int a, int b) {
    return (a + b) / 2.0;
}
```

Functions

- Functions must be defined **before** they are used
- Otherwise, the compiler doesn't know they exist yet
- (Later we'll learn how to fix this with declarations)

Correct Order

```
int add(int a, int b) {
    return a + b;
}

int main() {
    std::cout << add(2, 3) << std::endl;
}
```

Functions

- Functions help us:
  - Avoid repeating code
  - Give names to ideas
  - Make programs easier to debug
- If you copy and paste code more than once:
  - It probably should be a function

Questions?

Quiz

End of day 2

In Class Exercise

- Write a function called `square`
- It takes an integer as input
- It returns the square of that number
- Call it from `main` and print the result

In Class Exercise

Example Solution

```
#include <iostream>

int square(int x) {
    return x * x;
}

int main() {
    std::cout << square(5) << std::endl;
    return 0;
}
```

In Class Exercise

- Write a function called `divide`
- It takes two doubles as input
- It returns one number divided by the other
- Call it from `main` and print the result

In Class Exercise

Example Solution

```
#include <iostream>

double divide(double a, double b) {
    return a / b;
}

int main() {
    std::cout << divide(10, 2) << std::endl;
    return 0;
}
```

Functions

- Sometimes, we want a function to have a "default" value for a parameter
- This allows the function to be called with fewer arguments
- These are called **default arguments**

Default Arguments

- A default argument is a value that is used if no argument is provided
- Default arguments are specified in the function definition
- They use the assignment operator =

Example

```
#include <iostream>

void num_students_in_class(int students = 0) {
    std::cout << "There are " << students << "in this class"
        << std::endl;
}

int main() {
    num_students_in_class();
    num_students_in_class(100);
    return 0;
}
```

Default Arguments

- If we call the function without an argument:
  - The default value is used
- If we provide an argument:
  - The default value is ignored
- The function body does not change

Default Arguments

- Default arguments must come **after** non-default arguments
- This is because C++ matches arguments from left to right

Valid

```
int add(int a, int b = 5);
```

Invalid

```
int add(int a = 5, int b); // ERROR
```

Default Arguments

- You can have multiple default arguments
- But once one parameter has a default:
  - All parameters to the right must also have defaults

Example

```
int compute(int a, int b = 10, int c = 20);
```

Scope

- Now let's talk about **scope**
- Scope determines:
  - Where a variable exists
  - Where it can be accessed
- When a variable goes out of scope:
  - It no longer exists

Scope

- The most common scope we've seen so far is **function scope**
- Variables declared inside a function:
  - Only exist inside that function

Example

```
#include <iostream>

void my_function() {
    int x = 10;
    std::cout << x << std::endl;
}

int main() {
    my_function();
    // std::cout << x << std::endl; // ERROR
    return 0;
}
```

Scope

- Parameters are also scoped to the function
- They only exist inside the function body

Example

```
void print_value(int value) {
    std::cout << value << std::endl;
}

// value does NOT exist here
```

Scope

- Scope is created by **curly braces** {}
- This is not limited to functions
- Any pair of braces creates a new scope

Block Scope

- A block is any code wrapped in {}
- Variables declared inside a block:
  - Only exist inside that block

Example

```
#include <iostream>

int main() {
    {
        int x = 42;
        std::cout << x << std::endl;
    }

    // std::cout << x << std::endl; // ERROR
    return 0;
}
```

Scope Shadowing

- You can declare a variable with the same name in an inner scope
- This is called **shadowing**
- The inner variable temporarily hides the outer one

Example

```
#include <iostream>

int main() {
    int value = 10;

    {
        int value = 20;
        std::cout << value << std::endl;
    }

    std::cout << value << std::endl;
    return 0;
}
```

Scope

- Scope helps prevent:
  - Name collisions
  - Accidental misuse of variables
- It also helps the compiler:
  - Know when memory can be reclaimed

Common Scope Errors

- Using a variable outside of its scope
- Assuming variables live forever
- Reusing variable names unintentionally

Questions?

Hello World Program Revisited

- At this point, we know almost all parts of our hello world program
- We understand how functions work, and how the main function works
- We understand character arrays and C-style strings for "Hello World"
- We understand how printing to the character output works and how to add a new line
- We understand scope
- But there's still a few missing pieces here...
  - What is std::?
  - What is what does including iostream do?

Local Development

- To be able to successfully answer these questions, we need to learn how to develop locally
- This is extremely critical - unless we can develop on our own computer, we will have a tough time distributing our code
- Additionally, as a C++ developer, we want to be able to write and run code on any server, not just our computer
- To do this, we will not only code on our computer, we will do so in the terminal
  - And we'll edit our code using a terminal editor, just like in those hacker movies
- Lack of command line/terminal knowledge will be a big red flag when you go to get hired, so we're going to do all development in the terminal from here on in to build these skills

Local Development

- To reiterate: using the terminal is a requirement of this course
- Not using one will hurt you when it comes to exams
- The course project will have strange permission issues if you aren't using the terminal
- We will use the terminal in conjunction with git, GitHub, PyTest, etc
  - If you don't use the terminal for these, your grades will suffer
- Some students last semester found wild workarounds for this, like using the pseudo terminal in VS code
  - This is not the terminal
  - They could not complete the later assignments
- If you can't figure this stuff out, you have to come and talk to me or the TAs and get help
- And just to repeat: It'd be tough to get hired without this terminal knowledge

Linux

- Your computer consists of a bunch of hardware
  - Your keyboard
  - Your speakers
  - Your screen
  - etc
- At a high level, your operating system is the code that lets you interact with the hardware

Linux

- For windows users, Windows is the operating system that is interacting with the hardware
- How many people are currently using windows (non-apple, non-linux)?
- I bring this up just to let you know, that is a bit of a red flag when applying to jobs
  - Why?

Linux

- Well, there is another type of operating system that is not windows called Linux
- Macs (from apple), Ubuntu computer, both run Linux-based operating systems
- Here are some other systems that also run Linux-based operating systems:
  - Iphone
  - Android
  - Mac
  - Ubuntu
  - Servers (web, databases, cloud)
  - Cars
  - General consumer electronics
  - And so much more... pretty much all linux
    - Except for Windows

Linux

- When you run windows, it's sort of an admission that you aren't routinely working on any of the above, not working on databases, self-hosting websites, etc
  - The skills to run these things on windows are NOT transferable to running them on Linux, they are totally separate
- There are three ways to fix this:

Linux

- Option 1 (recommended for most): Use WSL2 (windows subsystem for linux)
  - This is good enough for beginners
  - It "emulates" Linux
  - Still, this will only last you your first few years, since WSL2 can't do everything Linux can do, and is a lot slower
  - Definitely the best place to start for beginners if you're stuck with a windows computer

Linux

- Option 2: Get a Mac (from apple) when you buy a new computer instead of Windows
  - macOS is Unix-based and behaves very similarly to Linux for development
  - Used m1 macs are quite fast and typically go for around 300 bucks
  - A brand new mac mini is 500 bucks with the student discount

Linux

- Option 3: Alternatively, you can dual-boot your computer to run both Ubuntu and Windows at the same time
  - This is very complicated though, and I don't recommend it anymore
  - Windows is like a virus that really doesn't want you running anything else
  - If you screw up, you "brick" your computer
    - That means your computer is only good for use as a brick because you have destroyed it so badly
  - If you do want to do this, my recommendation would be a Lenovo computer, as their drivers are usually very compatible, but any computer will do
  - I will also say, I don't personally know any serious developers that aren't running a linux distro like Ubuntu.
    - Long term, the people programming every day tend to end up here
    - You want maximum options and freedom
    - You want your computer to exactly match how programs are deployed on servers

Linux

- For mac users, if you go to settings, and then privacy and security, you want to enable your terminal to be able to run anything
- This will actually dramatically increase compilation times

The terminal

- I'll go over a few of the basics with you guys now
  - pwd (current dir)
  - ls (print files)
  - cd (change directory)
    - .. (up)
    - - (previous)
    - / (home)
  - mkdir (make directory)
  - rm (delete file)
  - rm -rf (delete dir)
  - cp (copy)
  - mv (move)

The terminal

- Continued
  - Up arrow (Bring back a prior command)
  - Tab (Complete a command)
  - ctrl + r (searching history)
  - ctrl c (kill whatever is running)
  - clear (clear the screen)

The terminal

- And many more! We don't have time to over them all in this course, but there are great tutorials online. I learned through `https://overthewire.org/wargames/bandit/`
- It will be a bit of a red flag if you want to be a systems programmer, but aren't good with the terminal, so practice!
- And beyond this... ask AI. Doesn't make sense to memorize syntax for anything beyond the basics, and for the average programmer, this is about it.
- To me these are all of the commands I typically use, and anything else I tend to look up using AI.
- There are some other commands I also use commonly, but I don't want to flood you guys all at once so we'll go over part 2 later on

Questions?

Vim/Emacs

- Vim/Neovim and Emacs are the two most dominant terminal editors
- Terminal editors are extremely useful when you need to work on a server (configuring an IDE to work on a server through an SSH tunnel is not easy or practical in many cases)
- Like all the other terminal skills, a terminal editor is a good tool to have in your toolbox, even if you don't use it daily
- Especially when it comes to high performance applications, there are many cases where you'll consistently be working in environments where a terminal editor will be the optimal solution.
- Linus Torvalds (Linux creator), Ken Thompson (co-creator of Unix, Go, B, etc), Richard Stallman (founder of GNU), Dennis Ritchie (co-creator of C, Unix) all used terminal editors
- I won't force you to use one (you can use an IDE like VS code), but you'll be missing out on a valuable skill, and vim motions can be transferred to IDEs as well

Vim

- Run `sudo apt-get update && sudo apt-get install vim` to install vim on WSL or Ubuntu
- For a mac, install homebrew (go to brew.sh) and then run: brew install vim
- Vim has four modes, we'll just go over the basics of three:
  - normal mode
  - insert mode
  - command mode

Vim: Insert mode

- Pressing the i key puts you in insert mode
- You can type anything you want like a normal editor
- press escape to leave insert mode and go back to normal mode

Vim: Normal mode

- Normal mode is the default mode
- You can move your cursor around with the arrow keys, or hjkl, but not the mouse
- There are vim commands to help you jump around faster
- ex: w jumps forward by a word, and b jumps backwards
- If you want to learn all the shortcuts, there is a good tutorial by ThePrimeagen: `https://www.youtube.com/watch?v=X6AR2RMB5tE`.
- The shortcuts to move around can also be transferred to any IDE for faster movement
- To start, this is honestly good enough

Vim: Command mode

- If you are in normal mode and press the colon, you enter command mode
- You can run commands here, but really simply:
- escape + colon + w + q to save and quit (or omit the q to just save)
- That's it for now!

vimrc example

- `~/.vimrc` is a config file that you can add which will make vim more powerful
- You can customize vim pretty much any way you'd like with this config (many examples online)
  - For example, a tab becomes four spaces
  - When you copy a line, it pulls it into your clipboard
  - Turn those annoying bell sounds off
- I've posted a relatively simple one I use personally in huskyct
- Remember, this goes in your `~/.vimrc` location.
- It's important to have a vimrc to convert tabs to spaces.
  - Spaces have the same spacing in every editor whereas tabs display differently
  - Most autoformatter tools convert everything spaces, and many languages even require spaces now
  - With a vimrc, tabs get autoconverted to spaces so you get the best of both worlds

vimrc example

- To clarify, take the posted example:
- Run vim `~/.vimrc`
- Press i to enter insert mode
- Copy paste my config (or another config you find from google/AI) into that file
- Press escape, the colon character, w, q, enter, to save and quit

Questions?

Compiling and Running our First C++ Program

- Now that we understand the basics of C++, we can compile and run programs locally
- C++ is a **compiled language**
- This means our source code must be translated into machine code *before* it can run
- This translation is done by a **compiler toolchain**

Installing the C++ Toolchain (Linux / WSL / Mac)

- First, make sure your system is up to date for WSL/Ubuntu:
  - `sudo apt-get update`
  - `sudo apt-get upgrade`
- For a mac, instead run: brew install gcc
  - This will also update this to the latest version of gcc, which we need.

Installing the C++ Toolchain (Linux / WSL / Mac)

- This installs:
  - The C++ compiler (usually g++)
  - The linker
  - The C++ standard library

Source Files

- C++ source files typically end in `.cpp`
- Example:
  - `hello_world.cpp`
- Header files (covered later) typically end in:
  - `.h` or `.hpp`

Compiling a C++ Program

Compilation Command

```
c++ hello_world.cpp -o program.out -std=c++23
```

- c++ is a **compiler driver**
- On most Linux systems, this invokes g++
- Some systems may be configured to use clang++

Breaking Down the Compilation Command

- `hello_world.cpp`
  - The source file
- `-o program.out`
  - Names the output executable
  - If omitted, the default output is `a.out`
- `-std=c++23`
  - Specifies the C++ language standard

Running an Executable

Run Command

```
./program.out
```

- . means the current directory
- Linux does **not** search the current directory by default
- The slash specifies the exact file to execute

What Is an Executable?

- An executable is a binary file containing machine instructions
- It is produced by the **linker**
- It must:
  - Contain valid machine code
  - Have execute permissions

Inspecting Files

```
file program.out
ls -l program.out
```

Compilation Pipeline Overview

- C++ compilation happens in multiple stages:
  1. Preprocessing
  2. Compilation
  3. Linking
- The c++ command usually performs all stages automatically

Preprocessing

- Happens *before* compilation
- Handles:
  - `#include`
  - `#define`
  - Conditional compilation
- `#include <iostream>` literally copies text
- It is **not** a library import

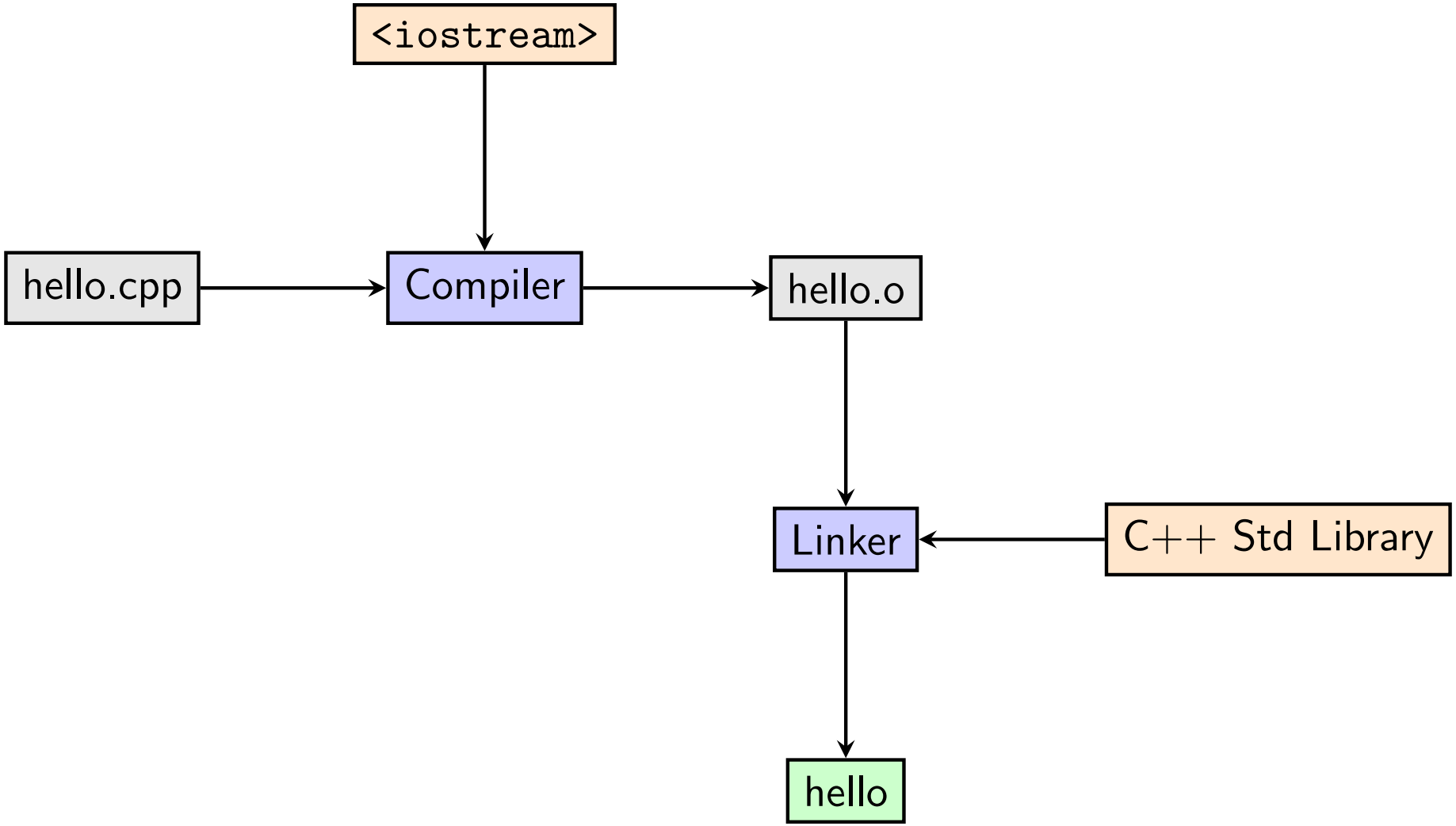
Compilation

- The compiler:
  - Checks syntax
  - Checks types
  - Translates code into machine instructions
- Output: an **object file** (`.o`)
- Object files are *not executable*

Linking

- The linker:
  - Combines object files
  - Resolves function and variable references
  - Links the C++ standard library
- Output: the final executable

C++ Compilation Pipeline (Visual)



Separating Compile and Link Steps

Compile Only

```
c++ -c hello.cpp -o hello.o -std=c++23
```

Link

```
c++ hello.o -o hello
```

- Useful for large projects
- Only recompile files that changed

Three Kinds of Errors

- **Compile-time errors**
  - Syntax errors
  - Type errors
- **Link-time errors**
  - Undefined references
- **Run-time errors**
  - Crashes
  - Undefined behavior

Debug vs Release Builds

Debug Build (Always Use for Development)

```
c++ hello.cpp -g -O0 -std=c++23
```

Release Build

```
c++ hello.cpp -O3 -std=c++23
```

- Debug builds are slower but debuggable
- Release builds are fast but hard to debug

Sanitizers (Extremely Important)

- Sanitizers detect bugs *while your program runs*
- They catch:
  - Out-of-bounds array access
  - Use-after-free
  - Integer overflow
  - Undefined behavior

Recommended Student Build

```
c++ hello.cpp -g -O0 -fsanitize=address,undefined -std=c++23
```

Important Optimization Warning

- Optimizers assume undefined behavior never happens
- They may:
  - Remove code
  - Reorder instructions
  - Produce surprising results
- Never debug optimized code
- Always debug with:
  - `-O0 -g -fsanitize`

Recap

- C++ programs must be compiled before running
- Compilation has multiple stages:
  - Preprocessing
  - Compilation
  - Linking
- Executables are machine code binaries
- Debug, release, and sanitized builds behave differently

Day 3 Lab

- What did people think about the bandit game?

Tmux

- Tmux is another useful tool
- You can open multiple windows
- It remains open on a server (and can be running your programs), even after you disconnect
- This allows for no ssh flakiness, and gives you the ability to run long programs

Tmux

- On Linux:
  `sudo apt-get update && sudo apt-get install tmux`
- On mac: brew install tmux (you may already have it!)

Tmux

- type tmux to enter a tmux session
- Whenever you type a command, it's ctrl+b, a very breif pause, and then the next key
- ctrl+b " to split the panes top to bottom
- ctrl + b % splits them left to right
- ctrl + b `<arrow_key>` to move around
- ctrl + b d to detach
- tmux attach to reattach
- Many more useful commands, but that's it for now. A useful cheatsheet `https://gist.github.com/MohamedAlaa/2961058`

Questions?

#include in C++

- #include is a macro. Before the C++ compilation starts, the #include tells the C++ preprocessor to copy the entire contents of the other file into your source code.
- This includes .h files, which are the declarations for functions and classes (the contract)
  - .h files are the contract, .cpp files are the implementation of the contract
  - You can (and should) omit the .h from the standard library headers now

Include Example

```
#include <iostream>
```

#include in C++

- Where to pull from
  - <> indicates standard headers (pull from standard library, skip user paths)
  - "" indicates user headers (pull from user paths first, then standard library)
- Looks in
  - Standard library search paths
  - User provided paths
    - Can use -I pathname to indicate where to search during compilation (we'll see this later)
- Caveat: Must protect against multiple inclusions (we'll come back to this)

Include Example

```
#include <iostream>
```

header files in C++

- So what do these headers look like?
- Well, typically they end in either .h (C) or .hpp (C++)
- They declare the contract that is implemented in the C++ files
- Let's take a look with some of our code

header files in C++

- This is code that calls a function
- But remember, the compiler must see a declaration of a function before it's called
  - This is unlike other languages like Python
- We must write the function out beforehand if we don't first declare it

Example

```
#include <iostream>

void my_function(int x) {
    std::cout << x << std::endl;
}

int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
```

header files in C++

- Much like we can declare an integer before assigning it, we can declare a function to get implemented later on
- To do this, we just write the function stub, and we can write the implementation afterwards
- Now C++ knows what the function looks like, and this code runs just fine!
- This declaration can live in a header so multiple files can agree on the same function signature

Example

```
#include <iostream>
void my_function(int x);
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
```

header files in C++

- However, we can further separate out our function declarations, the contract, into header files
- Let's go ahead and move our function stub into a header file called func.hpp

Example

```
void my_function(int x);
```

header files in C++

- Our code runs just the same!
- Remember:
  - Brackets indicate standard library
  - Double quotes indicate a user defined library

Example

```
#include <iostream>
#include "func.hpp"
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
void my_function(int x) {
    std::cout << x << std::endl;
}
```

header files in C++

- Still, this isn't great
- It'd be much nicer to have our implementation, our function, also in it's own file
- That way we can clearly separate our implementations from our main function, our entry point

Example

```
#include <iostream>
#include "func.hpp"
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
void my_function(int x) {
    std::cout << x << std::endl;
}
```

header files in C++

- So let's go ahead and move our function into a file called func.cpp

Example

```
#include <iostream>
#include "func.hpp"  // We include this to prevent
    accidental file changes later on for func.hpp
void my_function(int x) {
    std::cout << x << std::endl;
}
```

header files in C++

- Now we can compile and run our code, and it's looking pretty nice!

Example

```
#include <iostream>
#include "func.hpp"
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
```

header files in C++

- Now often times, especially when working on larger repos, you'll typically group files together
- You don't really want all of your headers and all of your cpp files in the same place
- So let's go ahead and group our files into:
  - An includes directory, where we'll have our .h and .hpp files
  - An src directory, where we'll have our .cpp files

header files in C++

- Now to compile our program, we can run c++ src/main.cpp src/func.cpp -I include
  - We must pass in the -I flag because we no longer have our header files within our current working directory
  - -I let's the compiler know "look for my header files here"
- Now we have a good working project structure! Looks great! Runs great!

header files in C++

- Bringing this home: This is what including iostream is doing!
- iostream is being included from the standard template library
  - Specifically the headers are being included
  - The compiler sees declarations from the header
  - The linker later connects your code to the standard library's compiled implementations
- Remember, brackets indicate standard library, double quotes indicate user library

Namespaces

- Namespaces are a way to group a lot of entities together
- You can put constants, variables, functions, etc in a namespace
- You can also nest namespaces!
- You can think of namespaces similar to how you interact with Python modules
  - But namespaces only affect naming at compile time
  - There is no overhead at runtime
- For example, in Python, you can import time, and call time.sleep

Namespaces

- In C++, we:
  - include iostream
  - iostream modifies the std namespace (allowed in C++!)
    - iostream does this by declaring names for classes, objects, functions, etc. inside the std
    - It does not modify an existing namespace at runtime
  - We can get functions and classes from the std namespace like std::cout
- C++ provides us with a lot of libraries that we can include, we'll go over some of these later

Hello World!

- What happens if we remove the std from our program?
- We will get a nice error message since cout is not in the global namespace.

Hello World Example

```
#include <iostream>
int main() {
    cout << "Hello, world!" << std::endl;
    return 0;
}
```

Hello World!

- We can bring all names from a namespace into the current scope
- This affects name lookup, not the variables lifetime

Hello World Example

```
#include <iostream>

int main() {
    using namespace std;
    cout << "Hello, world!" << endl;
    return 0;
}
```

Hello World!

- This is bad practice!
- Now we don't know what we just imported
- We may accidentally introduce a function like std::sort into our scope, and how would we even know? What if we also had a function named sort?
- This is not good and adds ambiguity
- This is similar to running from time import * in Python

Hello World Example

```
#include <iostream>

int main() {
    using namespace std;
    cout << "Hello, world!" << endl;
    return 0;
}
```

When to use namespaces globally

- We've seen the syntax for using namespace std globally
- However, this will create ambiguity with many function names that we don't see.
  - For example, std contains a function called sort
  - This is very similar in Python to the bad practice of `from <module> import *`
- Instead, we can just "import" what we need
  - `using std::cout;`
  - Similar to `from time import sleep`

Hello World Example

```
#include <iostream>
using std::cout;
using std::endl;
int main() {
    cout << "Hello, world!" << endl;
    return 0;
}
```

Namespaces

- We can also add our own namespaces
- Let's write a function that adds two numbers
- See also how we can shorten our std::cout and std::endl statement

Hello World Example

```
#include <iostream>
using std::cout, std::endl;

int add(int a, int b) {
    return a + b;
}
int main() {
    cout << "Hello, world!" << add(10, 20) << endl;
    return 0;
}
```

Namespaces

- We can create our own namespace
- Calling it like this will get an error, since foo is now in the namespace

Hello World Example

```
#include <iostream>
using std::cout, std::endl;
namespace MATHFUNCS {
    int add(int a, int b) {
        return a + b;
    }
}
int main() {
    cout << "Hello, world!" << add(10, 20) << endl;
    return 0;
}
```

Hello World With Namespaces

- Instead we can access this function through the namespace using the scope resolution operator (::)

Hello World Example

```
#include <iostream>
using std::cout, std::endl;
namespace MATHFUNCS {
    int add(int a, int b) {
        return a + b;
    }
}
int main() {
    cout << "Hello, world!" << MATHFUNCS::add(10, 20) <<
        endl;
    return 0;
}
```

The Scope Resolution Operator (::)

- The operator `::` is called the *scope resolution operator*
- It tells C++ exactly where a name lives
- Syntax: `scope::name`

Why Scope Resolution Matters

- C++ allows the same name to exist in multiple scopes
- Without `::`, the compiler may not know which one you mean
- `::` removes ambiguity

Revisiting `std::cout`

- `cout` is not a global variable
- It lives inside the `std` namespace
- `std::cout` means:
  - "the name `cout` inside the namespace `std`"

What #include <iostream> Does

- Including <iostream> declares names inside std
- Examples:
  - std::cout
  - std::endl
  - std::cin
- Nothing is added to the global namespace

The Global Scope Operator

- `::name` refers to the global namespace
- This is useful when names are hidden by local scopes

Example

```
int x = 10;
int main() {
    int x = 5;
    std::cout << ::x << std::endl; // prints 10
}
```

Hello World With Namespaces

- What about nesting? How does that work?

Hello World Example

```
#include <iostream>
using std::cout, std::endl;
namespace MATHFUNCS {
    namespace ADDFUNCS {}
        int add(int a, int b) {
            return a + b;
        }
    }
}
int main() {
    cout << "Hello, world!" << MATHFUNCS::add(10, 20) <<
        endl;
    return 0;
}
```

Hello World With Namespaces

- We declare namespaces the same way if we split them out into header and cpp files
- The namespace should be declared both in the .hpp and in the .cpp files where you declare and define add
- Let's walk through an example of this

Main

- Now we've covered almost all aspects of our hello world program (brief refresher)
- So what is this return 0?

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

What is `main`?

- `main` is a special function in every C++ program
- It is the program's *entry point*
- When your program starts running, execution begins in `main`

Why Does `main` Return an `int`?

- `main` returns an integer to the operating system
- This value is called the program's *exit status*
- It tells the OS whether the program succeeded or failed

What Does `return 0` Mean?

- By convention:
  - 0 means success
  - Non-zero values mean failure
- This convention is used across Unix, Linux, and Windows

Non-Zero Return Values

- You can return different values to indicate different errors
- Example:

Example

```
int main() {
    return 1;  // something went wrong
}
```

Non-Zero Return Values

- We can see this by running echo $? in the terminal
  - This gets the exit status of the last command that was run in the terminal
- This exit status is used in a lot of places
  - For example, if you call a subprocess in Python, typically you will check the exit status code and error out if it's not zero
  - Additionally, we can demonstrate this in the terminal by changing two commands together

Example

```
int main() {
    return 1;  // something went wrong
}
```

Non-Zero Return Values

- In our terminals we can change two commands together with `&&`
- So here we can do `./program.out && ./program.out` and our program.out will run twice
  - The caveat here is that the chain will "break" if the exit status code is anything other than zero

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 1;  // something went wrong
}
```

Non-Zero Return Values

- When we run `./program.out && ./program.out`, Hello World is printed only once because the exit status code is a 1, and not a zero
- Fix our program to return a 0, and run it again
  - Now hello world prints out twice
- Another example of chaining: Compiling and running on the same line

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 1;  // something went wrong
}
```

Do We Always Need `return 0`?

- In C++, reaching the end of `main` is equivalent to:
  - `return 0;`
- This is *special to `main`*
- But! We shouldn't do this, even if the compiler allows it.

Example

```
int main() {
    std::cout << "Hello World" << std::endl;
}
```

Why Write `return 0` Anyway?

- Makes intent explicit
- Clear to readers new to C++
- Consistent with older C and C++ code
- Common in teaching and professional codebases

Who Uses the Return Value?

- The operating system receives the return value
- Other programs can check it
- This is how scripts detect success or failure

`main` Is Just a Function

- `main` behaves like other functions:
  - It has a return type
  - It can return early
- The difference is that the caller is the operating system

Main

- And with that, we've covered every aspect of what you need to know for this hello world program
- This is also how all C++ programs are going to run, and the fundamentals of how they compile and run
- Now we'll close out a few of the finer details of this, and then we'll quickly start expanding what we can do in C++

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Questions?

End of day 4

#include in C++

- Let's dive back briefly into iostream
- #include is a macro. Before the C++ compilation starts, the #include tells the C++ preprocessor to copy the entire contents of the other file into your source code.
- This includes .h files, which are the definitions for functions and classes
  - .h files are the contract, .cpp files are the implementation of the contract
  - You can (and should) omit the .h from the standard library headers now

Include Example

```
#include <iostream>
```

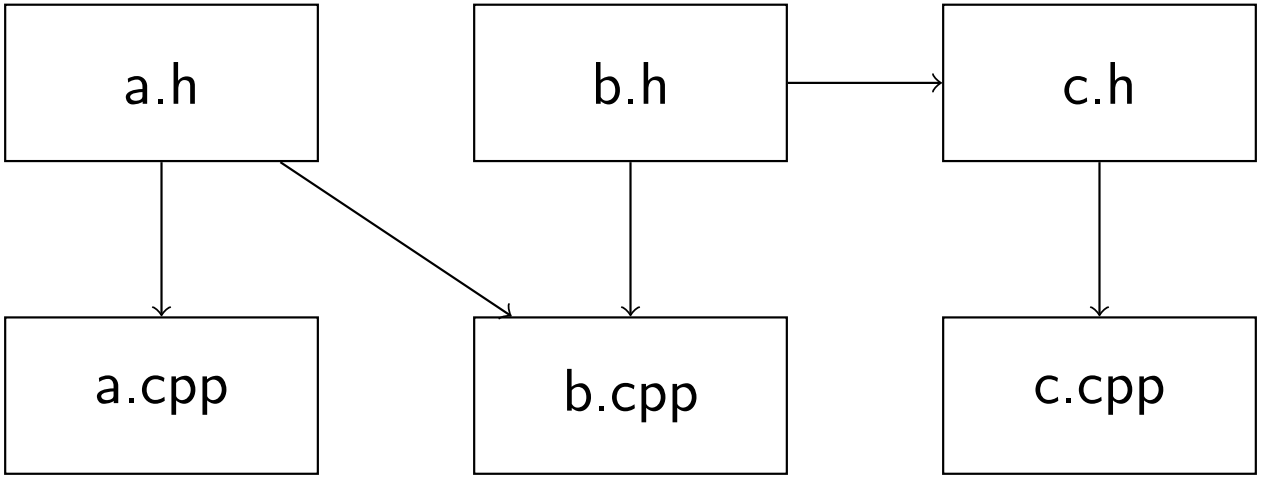
#include in C++

- Where to pull from
  - <> indicates standard headers (pull from standard library)
  - "" indicates user headers (pull from user paths)
- Looks in
  - Standard library search paths
  - User provided paths
    - Can use -I pathname to indicate where to search during compilation
- Caveat: Must protect against multiple inclusions

Include Example

```
#include <iostream>
```

Header and Source File Relationships



#include in C++

- Let's dive into this program we have here
- a.h, b.h, and c.h are going to be our header files for this program.
- These headers serve as the contract, and the .cpp files serve as the implementation of the contract

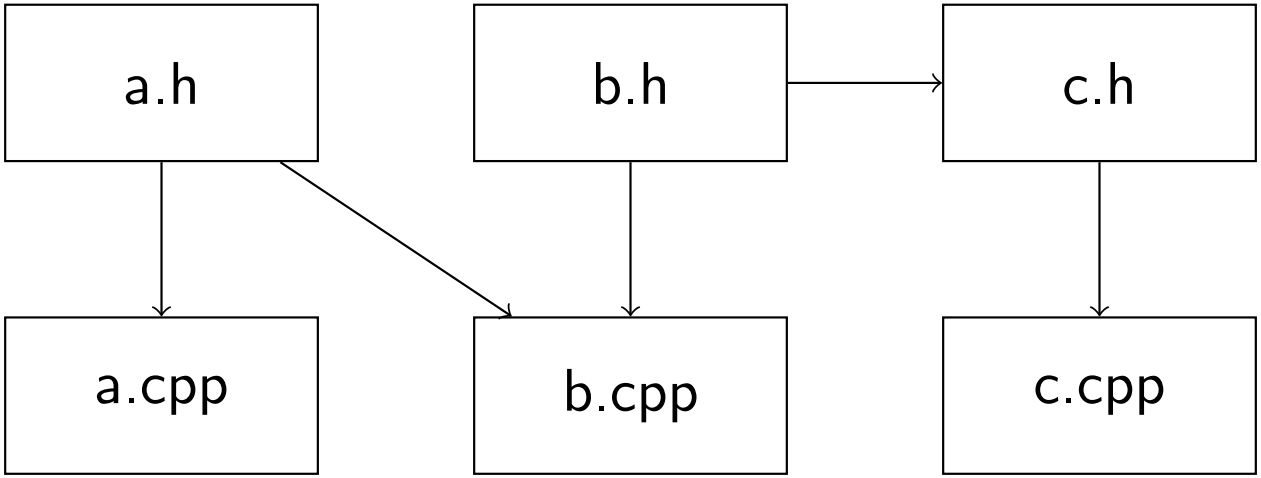
#include in C++

- So for example, a.cpp is the implementation of the contract defined in a.h
- Likewise b.h defines the contract that b.cpp implements
- The same is also true for c.h and c.cpp

#include in C++

- I've gone ahead and included these relationships within the diagram of how these files rely on each other
- So b.cpp RELIES on a.h. In our specific example, b.cpp is calling functions from a.h
- Because b.cpp relies on a.h and calls functions declared in a.h, the program will not compile unless we include the a.h function headers in b.cpp

Header and Source File Relationships

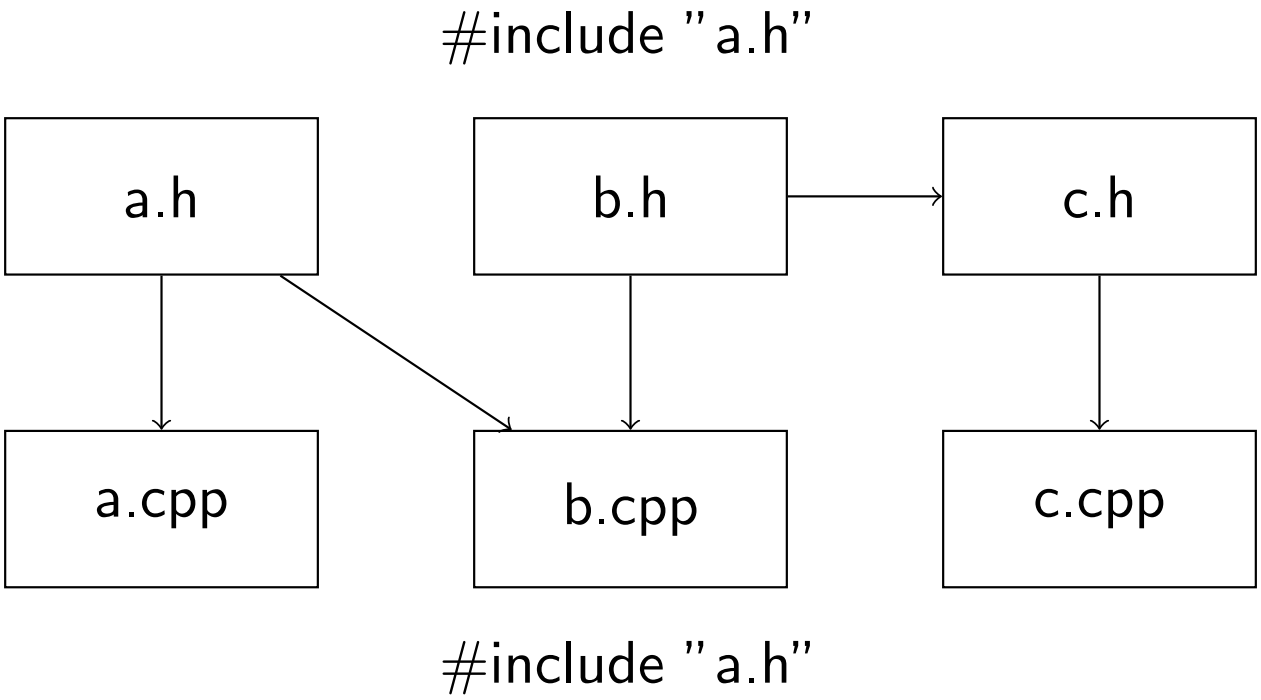


#include "a.h"

`#include` in C++

- So let's go ahead and make sure to incldue `#include "a.h"` in b.cpp.
- Next up is c.h
- In our example c.h is going to call functions declared in b.h
- Because of this we must make sure to include these in c.h as well, as shown here:

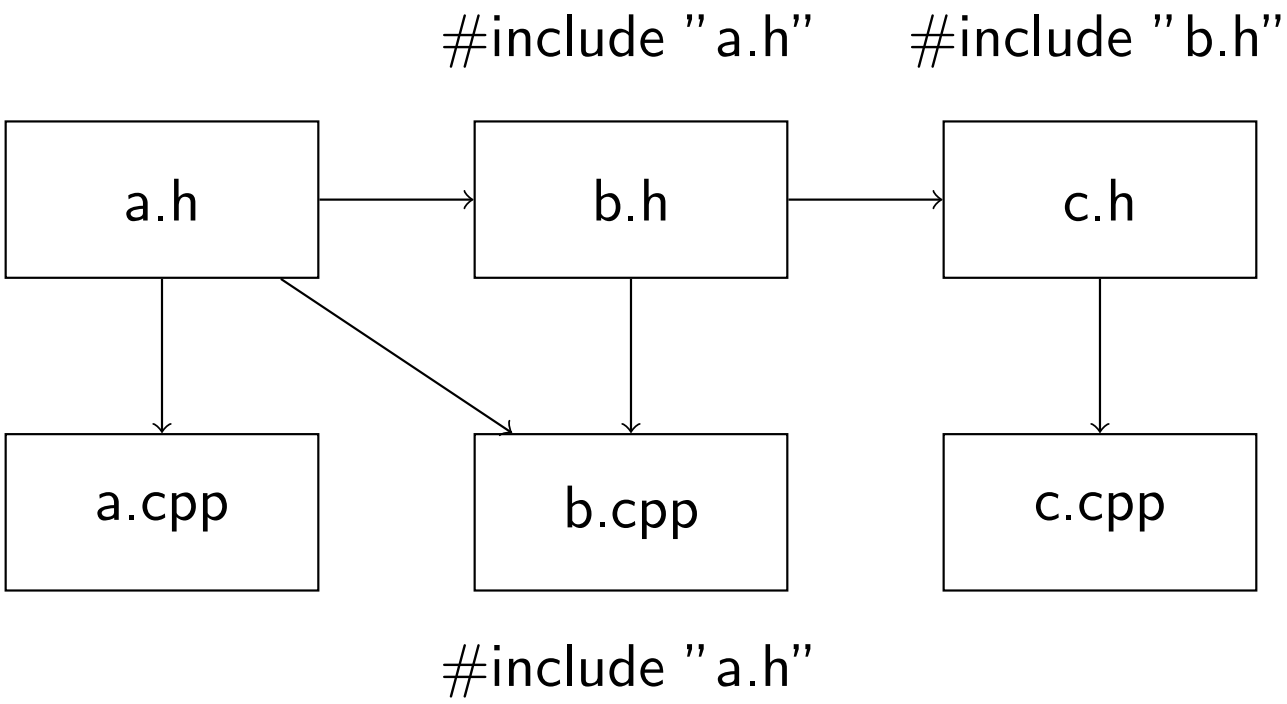
Header and Source File Relationships



`#include` in C++

- Hmm, this is not quite right.
- b.cpp calls functions declared in a.h, so should we add a `#include "a.h"` to c.h?
  - No!
  - We should instead include a.h in b.h.
  - That way include a.h will be copied everywhere that b.h is used automatically.
  - This is much cleaner than manually copy pasting include a.h everywhere that is using b.h.

Header and Source File Relationships



- a.h gets included twice!

#include in C++

- Now that we've included everything properly, we can compile this example program
- Remember that the preprocessor and compiler starts by:
  - Copying the declarations in a.h into a.cpp
  - And writes the output to a.o

#include in C++

- Next the preprocessor and compiler will move to b
- First b.cpp includes a.h since b.cpp is calling functions from a.h
- So the preprocessor will copy a.h into b
- Then the preprocessor moves to copy b.h into b
- But wait! b.h also includes a.h
- The preprocessor is now copying a.h into b twice, and declaring the same functions twice!
- This is terrible!

#include in C++

- Keep in mind: This happens even though our graph has no cycles!
- This double inclusion problem still occurs even with a directed acyclic graph
  - Directed: Arrows go in one direction
  - Acyclic: Has no cycles (has no loops)

#include in C++

- So what about a.h?
- Can we just not include a.h in b.cpp since a.h is already included in b.h?
- No!
- b.cpp calls functions from a.h so we must include a.h itself
- Relying on other headers downstream to include things properly is very bad practice
  - So we should just include a.h wherever we are using it
- Even if you use the a.h included in b.h, it's fragile code:
  - a.h could at some point be removed from b.h
  - Then your code would break!

#ifndef

- Use ifndef for every header to avoid this. Example below for b.h.
- a.h looks exactly the same.
- A_H is a name we make up, we can select it, but this is generally how it's done (FILENAME_H)
- (explanation of ifndef)

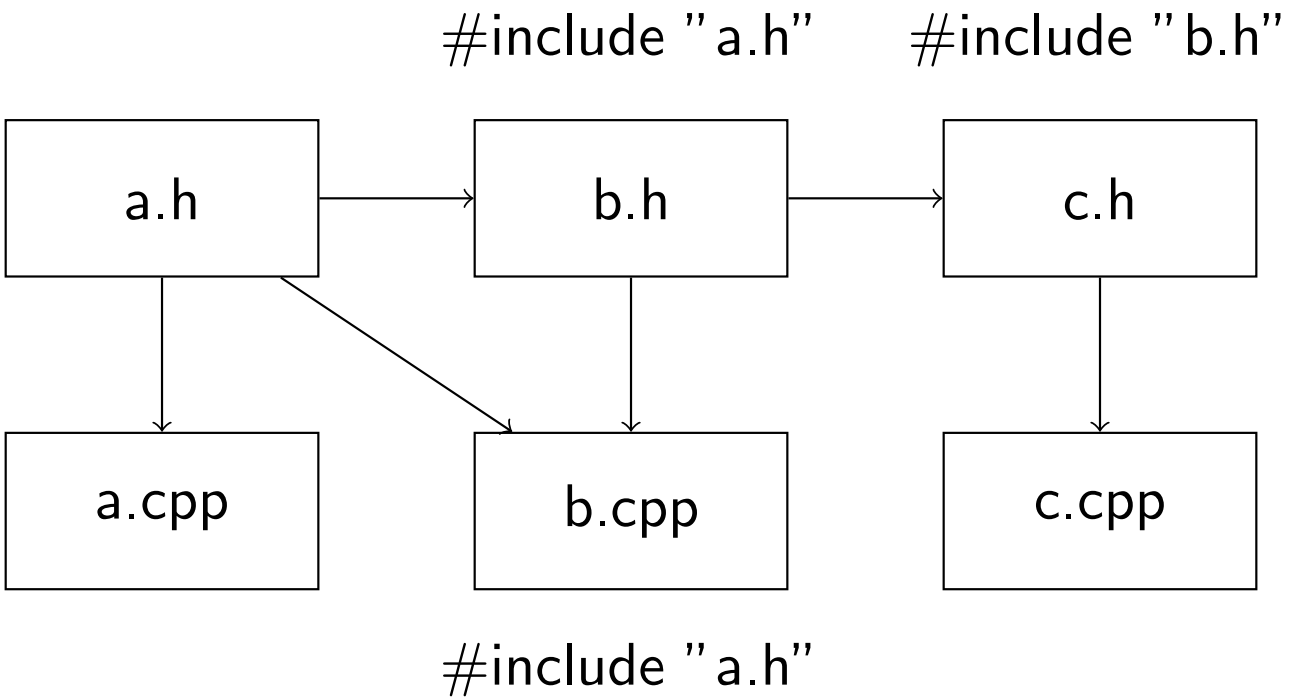
a.h

```
#ifndef A_H
#define A_H
//Insert a.h code here
#endif
```

b.h

```
#ifndef B_H
#define B_H
//Insert b.h code here
#endif
```

Putting it all together



a.h

```
#ifndef A_H
#define A_H
//Insert a.h code here
#endif
```

#include in C++

- So let's bring this concept home
- I've omitted b.h and c.h for space, but they look similar to a.h (just replace the letters with b or c).
- Compilation starts at a.cpp
- a.cpp will then pull in a.h.
- The preprocessor will go through the if not defined (and it is not defined), so it will define a.h, and then end if.
- This initial process will have now created a.o successfully

#include in C++

- Next the preprocessor and compiler will start on b
- b.cpp includes a.h
- a.h will not yet be defined for this .o file, and so the compiler will define it
- The compiler will then include b.h
- b.h includes a.h, but a.h is already defined!
- Therefore, due to our ifndef statement, a.h is not added a second time
- The compiler just continues with the b.h definition, and hits that endif
- b.o compiles successfully and compilation continues.

#include in C++

- After a.o, b.o, and c.o are generated (i.e. all the .o files are generated), the linker brings them together into an executable that we can use
- Easy peasy, we've fixed this double includes issue.
- Make sure to always do this for your header files, also known as header guards, also known as include guards.

Header Guards Summary

- That's how ifndef works, include it in every single header file
- Header guards are not enforced by the language itself
  - But they are good practice
- These are also a backwards compatible feature of the C language
  - Like much of C++, if you don't protect yourself and use these patterns, it will cause problems down the line!
  - Always use header guards!

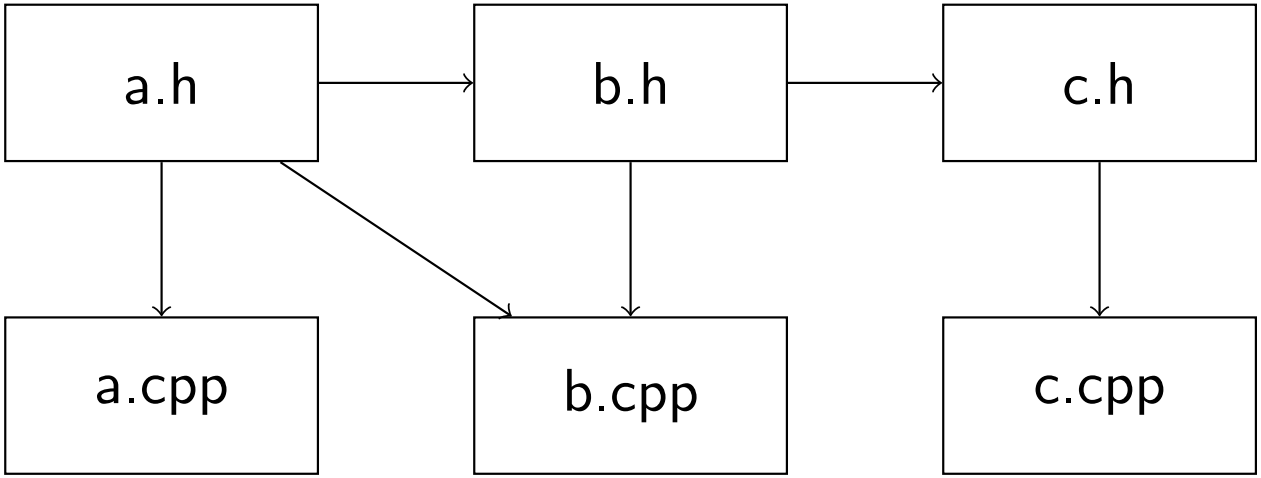
#pragma once vs include guards

- Standard, portable approach in C & C++: `#ifndef`/`#define`/`#endif` include guards.
- `#pragma once` is a widely supported *non-standard* alternative in C/C++.
- Pros: simple (one line).
- Cons: not in the ISO standards; some niche/older compilers may not support it.
- Modern compilers optimize traditional guards, so build-time differences are usually small.
- Used by several major tech companies, including Nvidia with CUDA.

a.h

```
#pragma once
// a.h contents
```

a.h, b.h, c.h example



Questions?

Standard Input: `std::cin`

- `std::cin` represents standard input (usually the keyboard)
- It lives in the `std` namespace
- It is an input stream, just like `std::cout` is an output stream

The Extraction Operator (>>)

- Input uses the extraction operator >>
- Syntax:
  - `std::cin >> variable;`
- This reads input from the user into a variable

Reading Input with std::cin

Example

```
#include <iostream>

int main() {
    int age;
    std::cout << "Enter your age: ";
    std::cin >> age;
    std::cout << "You entered " << age << std::endl;
    return 0;
}
```

Chaining Input Operations

- Like <<, the >> operator can be chained

Example

```
int x, y;
std::cin >> x >> y;
```

Printing Newlines

- There are two common ways to print a newline:
  - `std::endl`
  - `'\n'`

What Is the Output Buffer?

- Output written to `std::cout` is *buffered*
- This means output is stored temporarily in memory
- The program does not write to the screen immediately
- Instead, characters are collected in an output buffer

Why Output Is Buffered

- Writing to the screen is relatively slow
- Writing one character at a time is inefficient
- Buffering allows many characters to be written at once
- Result:
  - Faster programs
  - Smoother output

When the Output Buffer Is Flushed

- The output buffer is flushed when:
  - The buffer becomes full
  - The program finishes execution
  - You explicitly flush it (e.g., `std::endl`)
- Flushing forces output to appear immediately

What `std::endl` Does

- Prints a newline
- Flushes the output buffer
- Flushing forces output to appear immediately

What \n Does

- Prints a newline character
- Does *not* flush the buffer
- Usually faster than `std::endl`

`std::endl` vs `\n`

- Prefer `\n` for normal output
- Use `std::endl` when you *need* to flush
  - Debugging
  - Prompting before input

Whitespace in Input

- The >> operator:
  - Skips leading whitespace
  - Stops reading at the next whitespace
- Whitespace includes:
  - spaces
  - tabs
  - newlines

Whitespace Example

Example

```
int x;
std::cin >> x;
```

- Input: 42: OK
- Input: 42: OK
- Input: 42 99: Reads 42, leaves 99

A Common Input Pitfall

Example

```
#include<string>
int age;
std::string name;
std::cin >> age;
std::getline(std::cin, name);
```

- name may be empty!
- The newline from the previous input is still in the buffer
  - The newline gets eaten up by the getline
- This is a brief intro into strings, we'll come back to this later

Fixing Input Buffer Issues

Example

```
int age;
std::string name;

std::cin >> age;
std::cin.ignore();
std::getline(std::cin, name);
```

- `std::cin.ignore()` removes leftover characters
- Now `getline` works as expected

Input Rule of Thumb

- >> is good for:
  - numbers
  - single words
- `std::getline` is good for:
  - full lines
  - input with spaces

Streams Are Stateful

- Input and output streams remember what happened before
- Leftover characters stay in the buffer
- This is why input order matters

Control Flow

- Next we'll review C control flow
  - If
  - While
  - for
  - switch
  - etc

Branching with if

- Same as in C
- If there is a single instruction, you can omit the brackets

Example

```
int main() {
    if(expression_or_condition){
        //code
    } else {
        // code
    }
}
```

Comparison Operators

- Comparison operators compare two values
- The result of a comparison is always a `bool`
  - `true` or `false`

Operator	Meaning
==	equal to
!=	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

Comparison Operator Examples

Example

```
int x = 5;
if (x == 5) {
    // true
}
if (x != 3) {
    // true
}
if (x > 0) {
    // true
}
if (x <= 10) {
    // true
}
```

Logical Operators

- Logical operators combine boolean expressions
- They are commonly used in `if`, `while`, and `for` conditions
  - We'll cover while and for loops soon

Operator	Meaning
&&	logical AND
||	logical OR
!	logical NOT

Logical Operator Examples

Example

```
bool correct_password = true;
bool has_2fa = false;

if (correct_password && has_2fa) {
    // user is allowed to log in
}

if (!correct_password || !has_2fa) {
    // login denied
}
```

Combining Comparisons and Logic

- Comparisons produce booleans
- Logical operators combine those booleans

Example

```
int score = 85;

if (score >= 70 && score <= 100) {
    std::cout << "Passing score" << std::endl;
}
```

Operator Precedence (Basics)

- Operator precedence determines the order expressions are evaluated
- You do **not** need to memorize everything
- Just remember the most common cases

High to Low Precedence (Common)
`!`
`*, /, %`
`+, -`
`< <= > >=`
`== !=`
`&&`
`||`

Operator Precedence Example

Example

```
int x = 5;

if (x > 0 && x < 10) {
    // evaluated as:
    // (x > 0) && (x < 10)
}
```

- Comparison operators run before logical operators
- Parentheses make intent explicit and improve readability

When in Doubt, Use Parentheses

- Parentheses () override precedence rules
- They make code:
  - Easier to read
  - Less error-prone

Example

```
if ((x > 0 && x < 10) || x == 42) {
    std::cout << "Valid value" << std::endl;
}
```

Branching with if

- Same as in C
- If there is a single instruction, you can omit the brackets

Example

```
int main() {
    int x = 5;
    if(x == 6){
        //code
    } else if (x == 5){
        // code
    } else {
        // code
    }
}
```

Conditions Must Be Boolean

- Every condition in control flow must evaluate to a `bool`
- This means the expression must be either:
  - `true`
  - `false`
- A very common bug is confusing assignment with comparison

Conditions Must Be Boolean

Common Bug

```
int x = 5;
if (x = 6) {   // WRONG: assignment, not comparison
    // always true!
}
```

Correct

```
if (x == 6) {  // comparison
    // runs only if x equals 6
}
```

Braces vs Single Statements

- If there is only a **single statement**, braces may be omitted
- However, this can easily cause bugs
- Only the *next statement* is controlled by the condition

Braces vs Single Statements

Buggy Code

```
if (x > 0)
    std::cout << x << std::endl;
    std::cout << "Done" << std::endl; // ALWAYS runs
```

Recommended

```
if (x > 0) {
    std::cout << x << std::endl;
    std::cout << "Done" << std::endl;
}
```

Scope Inside Control Flow

- Curly braces {} create a new scope
- Variables declared inside control flow blocks:
  - Only exist inside that block

Example

```
#include <iostream>
int main() {
    if (true) {
        int x = 10;
        std::cout << x << std::endl;
    }
    // std::cout << x << std::endl; // ERROR: x is out of
        scope
}
```

Switch

- Same as in C
- Cases must be a constant expression evaluated at compile time

Example

```
#include <iostream>
using std::cout;
int main() {
    int day = 2;
    switch(day) {
        case 1:
            cout << "Monday";
            break;  // DONT FORGET! (Fall through)
        case 2:
            cout << "Tuesday";
            break;
        // days 3-7 omitted for space
        default: // For when none of the cases match
            cout << "Invalid day";
    }
}
```

Switch Fallthrough

- If `break` is omitted, execution continues into the next case
- This is called **fallthrough**
- Sometimes intentional, often a bug

Example

```
switch (x) {
    case 1:
    case 2:
        std::cout << "One or Two" << std::endl;
        break;
    default:
        std::cout << "Other" << std::endl;
}
```

While

- Same as in C
- Single line expressions don't need brackets

Example

```
#include <iostream>
using std::cout, std::endl;
int main() {
    // Case 1: With brackets (multiple statements)
    int count = 0;
    while(count < 3) {
        cout << "Count: " << count << endl;
        count++;
    }
    // Case 2: Without brackets (single statement)
    int i = 5;
    while(i > 0)
        cout << "i = " << i-- << endl;
}
```

Do-While

- Same as in C
- Single line expressions don't need brackets
- Executes at least once (body runs before condition check)

Do-While Loop Examples

```
#include <iostream>
using std::cout, std::endl;
int main() {
    // Case 1: With brackets (multiple statements)
    int count = 0;
    do {
        cout << "Count: " << count << endl;
        count++;
    } while(count < 3);
    // Case 2: Without brackets (single statement)
    int i = 5;
    do
        cout << "i = " << i-- << endl;
    while(i > 0);
}
```

For

- Same as in C
- Single line expressions don't need brackets
- Three parts: initialization, condition, increment

For Loop Examples

```
#include <iostream>
using std::cout, std::endl;
int main() {
    // Case 1: With brackets (multiple statements)
    for(int count = 0; count < 3; count++) {
        cout << "Count: " << count << endl;
        cout << "Additional statement" << endl;
    }
    // Case 2: Without brackets (single statement)
    for(int i = 5; i > 0; i--)
        cout << "i = " << i << endl;
}
```

Loop Variable Scope

- Variables declared in a `for` loop header:
  - Exist only inside the loop

Example

```
#include <iostream>
int main() {
    for (int i = 0; i < 3; i++) {
        std::cout << i << std::endl;
    }
    // std::cout << i << std::endl; // ERROR
}
```

Range-Based for Loop (C++)

- C++ provides a simpler loop for iterating over collections
- This is **not** available in C
- Syntax:
  - `for (type variable : collection)`

Example

```
#include <iostream>
int main() {
    int nums[4] = {1, 2, 3, 4};
    for (int x : nums) {
        std::cout << x << std::endl;
    }
}
```

break; continue

- Same as in C
- break: Breaks out of the innermost loop or switch
- continue: skip the remainder of the block and go to the next loop

break, continue

```
#include <iostream>
using std::cout, std::endl;
int main() {
    for(int i = 0; i < 10; i++) {
        if(i == 5)
            break;  // Exit loop when i equals 5
        cout << "i = " << i << endl;
    }
    for(int j = 0; j < 5; j++) {
        if(j == 2)
            continue;  // Skip when j equals 2
        cout << "j = " << j << endl;
    }
}
```

break vs continue vs return

- `break`
  - Exits the *nearest loop or switch*
- `continue`
  - Skips to the next iteration of the loop
- `return`
  - Exits the *entire function immediately*

break vs continue vs return

Example

```
void foo(){
    for (int i = 0; i < 10; i++) {
        if (i == 3)
            continue;  // skip 3
        if (i == 7)
            break;     // exit loop
        if (i == 8)
            return;    // ends function
        std::cout << i << std::endl;
    }
}
```

Nested Conditionals

- A conditional can appear inside another conditional
- This is called a **nested conditional**
- The inner condition only runs if the outer condition is true

Example

```
bool logged_in = true;
bool is_admin = false;

if (logged_in) {
    if (is_admin) {
        std::cout << "Admin access" << std::endl;
    } else {
        std::cout << "User access" << std::endl;
    }
}
```

Nested Conditionals with else if

- Many nested conditionals can be simplified
- `else if` avoids deep nesting
- Improves readability

Example

```
int score = 85;

if (score >= 90) {
    std::cout << "A" << std::endl;
} else if (score >= 80) {
    std::cout << "B" << std::endl;
} else if (score >= 70) {
    std::cout << "C" << std::endl;
} else {
    std::cout << "F" << std::endl;
}
```

Why Deep Nesting Is Dangerous

- Deep nesting makes code:
  - Harder to read
  - Harder to debug
  - Easier to get wrong
- This is sometimes called "arrow code"

Hard to Read

```
if (a) {
    if (b) {
        if (c) {
            if (d) {
                do_something();
            }
        }
    }
}
```

Flattening Nested Conditionals

- Often you can simplify nested conditionals
- Combine conditions using logical operators

Better

```
if (a && b && c && d) {
    do_something();
}
```

The Ternary Operator

- The ternary operator is a compact `if / else`
- It is the only operator in C++ that takes three operands

Syntax

```
condition ? value_if_true : value_if_false;
```

Ternary Operator Example

Example

```
int score = 85;

std::string result = (score >= 70) ? "Pass" : "Fail";

std::cout << result << std::endl;
```

Ternary vs if / else

- Use the ternary operator when:
  - The logic is simple
  - You are choosing between two values
- Avoid ternary when:
  - The logic is complex
  - Side effects are involved

Good

```
int max = (a > b) ? a : b;
```

Bad

```
(a > b) ? do_this() : do_that();
```

Ternary Operator Pitfall

- Ternary operators can be nested
- This quickly becomes unreadable

Avoid This

```
std::string grade =
    (score >= 90) ? "A" :
    (score >= 80) ? "B" :
    (score >= 70) ? "C" : "F";
```

Rule of Thumb

- Prefer clarity over cleverness
- If a conditional takes effort to understand:
  - Rewrite it
  - Add braces
  - Use `if / else`
- Ternary is a tool, not a requirement

goto

- No

Name spaces in class exercise

Going over our in class exercise

End of Day 5

What Is Git?

- Git is a **version control system**
- It tracks changes to files over time
  - Basically: like saving a file, but you can save your entire repo
- Allows you to:
  - Undo mistakes
  - Experiment safely
  - Collaborate with others
- Git runs **locally** on your computer

What Is Git?

- Git is basically the version control almost every company uses
- If you don't know git on your resume, it's pretty much an instant reject
  - It's sort of like saying "I don't know how to use email"
- This is an extremely important tool to learn
  - It's a common instant-fail interview question

What Is GitHub?

- GitHub is a **website** that hosts Git repositories (codebases)
- It allows you to:
  - Store code online
  - Share code with others
  - Collaborate on projects
- Git = tool GitHub = service

What Is GitHub?

- The other reason GitHub is important: commit logs
- You can view and track someone's commit logs over time
- Recruiters and automated resume checkers look at this
  - They see: Do you actually code?
  - Or do you just "think" about coding with theory and classwork?
- See: Github.com/jfuruness
- Note: You can turn on in your settings the ability to have this graph populate with commits on private repos (repositories)

Repositories

- A **repository** (repo) (codebase) is a project tracked by Git
- A repo contains:
  - Files
  - Commit history
  - Branches
- Repositories can be:
  - Local (on your computer)
  - Remote (on GitHub)

Commits

- A **commit** is a snapshot of your project
- Each commit has:
  - A unique ID
  - A message describing the change
- Commits should be:
  - Small
  - Frequent
  - Meaningful
- Think of it like a save point in a video game that you can go back to

Basic Git Workflow

1. Modify files
2. Stage changes
3. Commit changes
4. Push changes

Commands

```
git status
git add file.cpp
git commit -m "Describe the change"
```

Common Git Commands

Most Used Commands

```
git init            # create a repo
git status          # see repo state
git add             # stage files
git commit          # save changes
git log             # view history
git push            # push changes to github (if set up)
```

What Is a Branch?

- A **branch** is an independent line of development
- Branches let you:
  - Experiment without breaking main code
  - Work on features in parallel
- The default branch is usually called `main`

Why Branching Matters

- You should not develop directly on `main`
- Branches allow:
  - Safer development
  - Easier collaboration
  - Cleaner history
- If something breaks, `main` is still safe

Branching Commands

Working with Branches

```
git branch                  # list branches
git checkout -b feature-x     # create branch
git checkout feature-x     # switch branches
git checkout main
```

Merging Branches

- When a feature is complete, it can be merged
- Merging combines branch history
- Sometimes Git needs help resolving conflicts

Merge Example

Example

```
git checkout main
git merge feature-x
```

Local vs Remote Repositories

- Your computer has a **local repo**
- GitHub hosts a **remote repo**
- You sync them using:
  - push
  - pull

Push and Pull

Commands

```
git push
git pull
```

What Is a Pull Request?

- A **pull request (PR)** is a request to merge code
- Created on GitHub
- Used to:
  - Review code
  - Discuss changes
  - Run automated checks

Pull Request Workflow

1. Create a branch
2. Make commits
3. Push branch to GitHub
4. Open a pull request
5. Review and merge

Why Pull Requests Are Important

- Encourage code review
- Catch bugs early
- Improve code quality
- Standard practice in industry
- This is how EVERYONE collaborates and works
    - Again, a very common interview question

Rule of Thumb

- Commit early and often
- Write meaningful commit messages
- Use branches for features
- Never be afraid to make a branch

Pytest

- Python is another language that is closely related to C/C++
- A lot of times, Python is actually running C++ code under the hood to make it faster
- We'll use Python for our testing since it's a lot easier to use PyTest than it is to get into C++ testing at this stage
- We'll only touch briefly on this.
  - I'll provide you the tests, you won't have to write them (for now)
  - So we'll just cover how to run the tests, rather than write them, since this is a C++ course

The Python Ecosystem

- One of the best parts of the Python language is the Python ecosystem
- Unlike a lot of other languages, Python makes it very easy to develop and share code
- For pretty much anything that you might want to do, someone likely has already coded it up
- Often times, code you write, will just be connecting other people's tools!
- This saves a TON of time, as people have spent years and years building out tools that you don't have to write or debug (at least, hopefully you won't have to debug)

Terminology

- There are a few terms that are critical to know for this and you'll hear them all the time in industry
- The first is what's called a package
- When you want to share your code with someone else, you need to turn that code into a Python Package
- The word package and module are essentially interchangeable
- We want to use the PyTest package for testing

Pip

- Python gives us a tool to be able to install packages called pip
- pip goes and searches up whatever name we give pip
- It then tries to find that name in the Python Package Index, also known as PyPI, and install the package we are looking up!
- But... where does it install that package?
- This is where Python virtual environments come in!
  - We need to understand how Python virtual environments work, *before* we can use pip to install anything

Global Installs

- By default, pip will install code to our computer!
- This is also known as installing a module or package globally
  - Also known as installing a module/package directly onto the operating system
  - The operating system is what is running your computer
    - Such as Windows, MacOS, Ubuntu, etc.
- Unfortunately, installing a package/module globally is a very bad idea
- Often times these days, your computer will actually prevent you from global installs like this

Danger!

- Your computer has code that runs every time it turns on
- That code is called the operating system
  - The operating system has code for the display so that you can see things
  - There's code for the keyboard so that you can type things
  - There's code for pretty much everything that you do
- Thankfully, 99% of the time, your operating system works just fine, and you never have to worry about this!
  - Although you may be aware of some phenomena like the "blue screen of death" that windows sometimes has
  - This is when code in your operating system has a very bad bug

Danger!

- Now your operating system actually also uses Python!
- And your operating system also has modules and packages, just like your code!
- When you install a package globally, you could actually be overwriting a package that your operating system relies upon
  - Even if it's the same package, you could be installing a different version of it than the one your operating system uses to run your computer!
- In other words, by installing packages globally, you could be changing code that your operating system needs to function
- That's really not good!

Danger!

- Additionally, if you install a package globally, and the install is broken in some way... now what?
- Uninstalling or fixing that package becomes extremely difficult
- This is especially true when you're building your own Python packages
  - This is even more true if you do things like add C++ or Rust bindings to those packages
- But it can also be a problem for installing other people's packages as well

Virtual Environments to the Rescue!

- Fortunately, Python Virtual Environments fix all of these issues!
- That is why they are so critical
- A Python Virtual Environment is an isolated area that packages get installed to
- They don't conflict at all with what your operating system is using
- If an installation has a problem... you can simply delete it
  - And not have to worry about deleting part of your operating system
- Let's dive into how to make these

How to Make a Python Virtual Environment

- I'll be going over two ways here for how to make a Python Virtual Environment
- In the first way, I'll cover how to make a Python virtual environment using a terminal (Mac, Ubuntu, Windows with WSL2)

How to Make a Python Virtual Environment: Terminal

- If you are on a Mac or Apple computer, you should be all set here, as virtual environments come with the default Python installation
- Otherwise, there are two commands that you need to run before we get started

terminal

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```

How to Make a Python Virtual Environment: Terminal

- From here on in it's the same for everyone:
- Don't forget to upgrade pip!
  - IDEs like VS Code will automatically upgrade it for you
  - But if you are in a terminal, you must do this yourself!

terminal

```
python3 -m venv env
source env/bin/activate
pip3 install pip --upgrade  # don't forget this!
```

Terminal: How to Install a Package

- Now that we have activated our virtual environment, let's go ahead and install a package
- Let's install our testing library PyTest
  - PyTest is by far the most popular testing framework
  - There really is no reason to ever not use PyTest for Python tests

Terminal: How to Install a Package

- Go ahead and just run python3 -m pip install PyTest
- You have now installed PyTest in your local environment!
- To check this, you can run python3 -m pip list to see all of the packages that have been installed
- You can even type python3 to execute python commands, and go ahead and import PyTest, and then exit()
- Mission success! We can import it!

Terminal: Global Install Differences

- Now let's see how this is different from a global install
- We can deactivate our virtual environment by running deactivate
- Now you should see those parenthesis disappear
  - That means we are no longer using any virtual environments
- Let's go ahead and run python3 -m pip list
  - pytest is no longer there!
- Try importing pytest after running python3
  - We can't! That's because it's not installed globally!

Terminal: Starting over?

- Now what if our installation was broken somehow and we wanted to start over?
- We could simply delete the virtual environment with an rm -rf env
- Then we can run through the process of creating a virtual environment again
- Let's go ahead and do that now for some practice

Terminal: Using pytest

- One more time let's just install pytest real quick
- Then let's dive back into our code and actually use it!
- Let's just write a simple test.py file
  - Note: Pytest files must start with the name test
- And now we can run our code (with our environment activated) with python3 -m pytest!
- We'll use this to test a lot of our code throughout this course

test.py

```
def test_add():
    assert 1 + 1 == 2, "Fail"
```

Exam

- There will be an exam soon!
- Make sure to bring:
  - ID
  - Computer
  - Computer charger
- Cheaters will be given a zero for the course

Exam

- I recommend going over:
  - The slides
  - The in-class exercises
  - The quizzes
  - The homework
- Make sure you can do these problems without help in a timely manner!
- And I would do more studying outside of this

Exam

- The exam is going to be a similar format of the quizzes
- There will be some short answer questions
- There will be several coding problems worth the majority of the points on the exam
- This is designed like a basic C++ interview (without algorithms)
  - My goal is to align these tests with how most industry jobs will test this algorithms knowledge
  - Minus algorithms, since that isn't covered here

Function Overloading

- C++ allows multiple functions to share the same name
- This is called **function overloading**
- The compiler decides which function to call based on:
  - Number of arguments
  - Types of arguments
  - Order of arguments
- This decision happens at **compile time**

Why Function Overloading Exists

- Improves readability
- Lets you express the same *concept* for different types
- Avoids awkward naming like:
  - `add_int`
  - `add_double`
  - `add_float`
- Very common in the C++ standard library

Basic Function Overloading Example

Example

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int main() {
    std::cout << add(2, 3) << std::endl;
    std::cout << add(2.5, 3.1) << std::endl;
    return 0;
}
```

How the Compiler Chooses

- The compiler looks at the **function signature**
- A function signature includes:
  - Function name
  - Parameter types
  - Parameter order
- Return type is **not** part of the signature

Return Type Is NOT Enough

Invalid Overload

```
int foo(int x);
double foo(int x);  // ERROR
```

- The compiler cannot choose based on return type
- This results in a compile-time error

Overloading with Different Parameters

Example

```
#include <iostream>
void print(int x) {
    std::cout << "int: " << x << std::endl;
}
void print(int x, int y) {
    std::cout << "two ints: " << x << ", " << y << std::endl
        ;
}
int main() {
    print(5);
    print(3, 4);
    return 0;
}
```

Overloading Can Go Wrong

- Some overloads introduce ambiguity
- The compiler must find *exactly one* best match
- Otherwise: compile-time error

Ambiguous Example

```
void foo(int x);
void foo(double x);

foo(3);   // int or double?
```

Can You Overload `main`?

- **No — `main` cannot be overloaded**
- The C++ standard allows only specific signatures
- The operating system is the caller
- Overloading `main` would break program startup

Valid main Signatures

- C++ allows exactly two common forms of main

No Arguments

```
int main() {
    return 0;
}
```

With Arguments

```
int main(int argc, char* argv[]) {
    return 0;
}
```

Command-Line Arguments

- `argc` = argument count
- `argv` = argument vector
- These are passed in by the operating system
- Used to control program behavior from the terminal

Understanding argc and argv

- `argc` tells you how many arguments exist
- `argv` is an array of C-style strings
- `argv[0]` is always the program name

Using Command-Line Arguments

Example

```
#include <iostream>

int main(int argc, char* argv[]) {
    std::cout << "Argument count: " << argc << std::endl;
    for (int i = 0; i < argc; i++) {
        std::cout << "argv[" << i << "]: "
                  << argv[i] << std::endl;
    }
    return 0;
}
```

Running with Arguments

- Compile:

Compile

```
c++ main.cpp -o program
```

- Run:

Run

```
./program hello world 42
```

C Refresher... is over!

- Up until now, almost everything has hopefully been a refresher
  - The terminal
  - Pytest
  - All the basic C functionality
- We haven't even reviewed pointers yet, but we will do this later on
- Now we will start to dive deep into C++

Strings in C and C++

- C++ supports two kinds of strings:
  - **C-style strings** (from C)
  - **C++ strings** (`std::string`)
- You will encounter both in real-world code
- However, `std::string` is almost always preferred

C-Style Strings

- A C-style string is:
  - An array of `char`
  - Ending with a null terminator: \0
- This comes directly from the C language
- C++ still supports this for compatibility
- They have less overhead, and therefore are sometimes still used

C-Style String Example

Example

```
#include <iostream>

int main() {
    char name[6] = "Justin";  // BUG!
    std::cout << name << std::endl;
}
```

- Why is this broken?
- "Justin" needs **7** characters:
  - J u s t i n \0

C-Style String Problems

- You must manually manage memory size
- Easy to overflow buffers
- No automatic resizing
- Comparing strings is painful
- Many security bugs come from this

C++ Strings: `std::string`

- C++ provides `std::string`
- It lives in the `std` namespace
- Handles memory management for you
- Much safer and easier to use

Why `std::string` Is Easier

- Automatically manages memory
- Can grow and shrink dynamically
- Supports:
  - Assignment
  - Comparison
  - Concatenation
  - Length queries
- Much harder to misuse

Getting String Length

Example

```
std::string s = "Hello";
std::cout << s.length() << std::endl;
std::cout << s.size() << std::endl;
```

- `length()` and `size()` are identical
- Runs in constant time

String Concatenation

Example

```
std::string first = "Hello";
std::string second = "World";

std::string result = first + " " + second;
std::cout << result << std::endl;
```

- Much cleaner than C-style concatenation
- No manual buffers

Comparing Strings

Example

```
std::string a = "cat";
std::string b = "dog";

if (a == b) {
    std::cout << "Equal" << std::endl;
} else {
    std::cout << "Not equal" << std::endl;
}
```

- ==, !=, <, > all work
- Lexicographical (dictionary) ordering

Accessing Characters

Example

```
std::string word = "hello";

std::cout << word[0] << std::endl; // 'h'
std::cout << word[4] << std::endl; // 'o'
```

- Strings are indexable like arrays
- Indices start at 0

Modifying Strings

Example

```
std::string s = "hello";
s[0] = 'H';
s += " world";

std::cout << s << std::endl;
```

- Strings are mutable
- Automatically resize when needed

Reading Strings from Input

Single Word

```
std::string name;
std::cin >> name;
```

Full Line

```
std::string line;
std::getline(std::cin, line);
```

- Same input rules you saw earlier apply

C-Style vs std::string

Feature	C-Style	std::string
Automatic resizing	No	Yes
Memory safety	No	Yes
Easy concatenation	No	Yes
Easy comparison	No	Yes
Beginner friendly	No	Yes

Rule of Thumb

- Use `std::string` by default
- Only use C-style strings when:
  - Interfacing with C libraries
  - Required by low-level APIs
- Modern C++ code should prefer safety and clarity unless you're looking for extreme speed

Mixing String Types

- C++ supports:
  - `std::string`
  - C-style strings (`char*`, string literals)
- These types can interact — but with rules
- Some combinations work
- Some combinations do **not** compile

Adding a C-Style String to std::string

Works

```
#include <string>

std::string s = "Hello";
s = s + " world";
```

- " world" is a C-style string literal
- std::string knows how to absorb it
- Result is a new std::string

C-Style String on the Left

Also Works

```
#include <string>

std::string s = "world";
std::string result = "Hello " + s;
```

- This still works
- Why?
  - C++ converts the C-style string into a `std::string`
  - Then performs string concatenation

C-Style + C-Style Does NOT Work

Does NOT Compile

```
char a[] = "Hello ";
char b[] = "world";

auto result = a + b;  // ERROR
```

- This is NOT string concatenation
- a and b are pointers / arrays
- + is undefined for C-style strings

Why This Fails

- C-style strings are just memory addresses
- Adding them would mean:
  - Pointer arithmetic
  - Not string concatenation
- C has no built-in string type
- That's why C uses functions like:
  - `strcpy`
  - `strcat`

Fix: Convert to std::string

Correct Way

```
char a[] = "Hello ";
char b[] = "world";

std::string result = std::string(a) + b;
```

- Convert at least one side to std::string
- After that, concatenation works normally

Concatenation Rules (Memorize This)

- `std::string + std::string`: works
- `std::string + C-style string`: works
- `C-style string + std::string`: works
- `C-style string + C-style string`: fails

Iterating Over a String

- A `std::string` is a sequence of characters
- You can loop over it just like an array
- C++ provides a cleaner way: **range-based for**

Index-Based String Iteration

Example

```
std::string word = "hello";

for (size_t i = 0; i < word.size(); i++) {
    std::cout << word[i] << std::endl;
}
```

- Works
- But verbose
- Easy to make off-by-one errors

Range-Based for with Strings

Cleaner Version

```
std::string word = "hello";

for (char c : word) {
    std::cout << c << std::endl;
}
```

- No indices
- No bounds checks
- Much easier to read

Finding substrings with .find

- `string::find()` searches for a substring or character
- Returns the index (starting at 0) where it is found
- Returns `string::npos` if not found

Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;

int main() {
    string s = "hello world";
    size_t pos = s.find("world");
    if (pos != string::npos) {
        cout << "Found at position " << pos << endl;
    }
}
```

What is size_t?

- size_t is the safe type to use for lengths and indices, since it's guaranteed to hold the size of any object in memory.

size_t Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;

int main() {
    string s = "hello";
    size_t len = s.size();
    cout << "Length = " << len << endl;
}
```

.find: int vs size_t

- `string::find()` returns `size_t`
- If the substring is not found, it returns `string::npos`
- `npos` = the largest possible `size_t` value
- Using `int` works for small examples, but cannot truly represent `npos`

.find: int vs size_t

size_t Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;
int main() {
    string s = "hello";
    // Using size_t (recommended)
    size_t pos1 = s.find("world");
    if (pos1 == string::npos)
        cout << "Not found (size_t)" << endl;
    // Using int (sometimes seen in C++ tutorials - wrong)
    int pos2 = s.find("world");
    if (pos2 == -1) // overflowed value
        cout << "Not found (int)" << endl;
}
```

Extracting substrings with .substr

- `string::substr(pos, len)` returns a substring
- `pos` is the starting index
- `len` is the number of characters (optional; defaults to rest of string)

.substr Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;
int main() {
    string s = "Alice Smith";
    string first = s.substr(0, 5);      // "Alice"
    string last  = s.substr(6);         // "Smith"
    cout << first << " " << last << endl;
}
```

Checking if a String Contains Another

- A very common operation:
  - Does this string contain some text?
- Examples:
  - Searching commands
  - Parsing input
  - Validating user data

std::string::contains (C++23)

Example

```
#include <string>
#include <iostream>
int main() {
    std::string text = "hello world";
    if (text.contains("world")) {
        std::cout << "Found it!" << std::endl;
    }
}
```

- Introduced in C++23
- Very readable
- Returns a bool

How `contains` Works

- Checks whether a substring exists
- Case-sensitive
- Equivalent to:
  - `find(...) != std::string::npos`
- Prefer `contains` when available

Uppercase and Lowercase in C++

- Unlike some languages:
  - C++ does NOT have `string.toUpper()`
  - Or `string.toLower()`
- Instead, C++ uses character-based functions
- This gives more control (and more responsibility)

Character Case Conversion

- Provided by the C++ standard library
- Located in `<cctype>`
- Common functions:
  - `std::toupper`
  - `std::tolower`
- Operate on **single characters**

Uppercase and Lowercase: Two Approaches

- There are two common ways to change string case:
  - Build a new string (safe and simple)
  - Modify the string in place (more direct)
- Both are valid
- Which one you use depends on context, although in industry, side effects typically are not good

Uppercase (Build a New String)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
std::string upper = "";
for (char c : text) {
    upper += std::toupper(c);
}
```

- Iterates by value
- Original string is unchanged
- Very easy to reason about

Lowercase (Build a New String)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
std::string lower = "";
for (char c : text) {
    lower += std::tolower(c);
}
```

- Same pattern as uppercase
- Only the function changes

Uppercase (Modify In Place)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
for (size_t i = 0; i < text.size(); i++) {
    text[i] = std::toupper(text[i]);
}
```

- Uses indexing
- Directly changes the original string
- No references required

Lowercase (Modify In Place)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
for (size_t i = 0; i < text.size(); i++) {
    text[i] = std::tolower(text[i]);
}
```

- Same loop structure
- Modifies characters one at a time

Why Show Both Approaches?

- Building a new string:
  - Easier to understand
  - No side effects
- Modifying in place:
  - More efficient
  - Less memory usage

Strings and Numbers

- Very common task in real programs:
  - Read input as a string
  - Convert it to a number
- Also common in the other direction:
  - Convert numbers to strings for output
- C++ provides standard library tools for both

String to Number Functions

- Provided by the C++ standard library
- Located in `<string>`
- Common functions:
  - `std::stoi` (string → int)
  - `std::stod` (string → double)
  - `std::stol` (string → long)

Example: std::stoi

Example

```
#include <string>
#include <iostream>

int main() {
    std::string s = "42";
    int x = std::stoi(s);

    std::cout << x + 10 << std::endl;  // 52
}
```

Example: std::stod

Example

```
#include <string>
#include <iostream>

int main() {
    std::string s = "3.14";
    double d = std::stod(s);

    std::cout << d * 2 << std::endl;
}
```

Important Notes

- These functions expect valid numeric strings
- Examples of valid input:
  - `"42"`
  - `"-10"`
  - `"3.14"`
- Invalid input causes runtime errors
- We will handle errors later in the course

Number to String

- Converting numbers to strings is also common
- C++ provides:
  - `std::to_string`
- Works with:
  - `int`
  - `double`
  - other numeric types

Combining Numbers and Strings

Example

```
int score = 95;
std::string message =
    "Score: " + std::to_string(score);

std::cout << message << std::endl;
```

Command-Line Arguments Are Strings

- Arguments passed into `main` are:
  - C-style strings (`char*`)
- These are often numeric values
- We usually convert them to numbers immediately

Example: Integer Argument

Example

```
#include <string>
#include <iostream>

int main(int argc, char* argv[]) {
    if (argc < 2)
        return 1;

    int x = std::stoi(argv[1]);
    std::cout << x * 2 << std::endl;

    return 0;
}
```

Running the Program

Terminal

```
./program 21
```

- Output:
  - 42

Example: Two Numeric Arguments

Example

```
#include <string>
#include <iostream>
int main(int argc, char* argv[]) {
    if (argc < 3)
        return 1;
    int a = std::stoi(argv[1]);
    int b = std::stoi(argv[2]);
    std::cout << a + b << std::endl;
    return 0;
}
```